Bioinformatics Self-Study Curriculum

A 12-Block, Sprint-Based Program

Statistical Genetics · Multiplexed Assays · Unix · Python · R · AI Tools

Hands-on exercises with real data in every session

Designed for independent learners in genomics, biostatistics, and computational biology.

No instructor required — all resources are freely available or open-access.

Contents

1 How This Curriculum Works (Read This First) 3
1.1 The Three Rules . . . 3
1.2 How a Block Works . . . 4
1.3 Your Toolkit (Set Up in Block 0) . . . 4
1.4 Interleaving: Why This Curriculum Mixes Things Up . . . 5
1.5 How to Use AI as Your Study Partner . . . 5

2 Blocks 1–2: Unix Foundations 6
2.1 Block 1: The Command Line . . . 6
2.2 Block 2: Scripts, Environments & Git . . . 9
2.2.1 Can I move on? Checkpoint: . . . 13
2.2.2 Resources . . . 13

3 Blocks 3–4: Python for Genomic Data 14
3.1 Block 3: Core Python via Biology . . . 14
3.2 Block 4: Pandas, Plotting, and Bio Libraries . . . 18
3.2.1 Checkpoint: . . . 20
3.2.2 Resources . . . 20

4 Blocks 5–6: R, Bioconductor & Differential Expression 21
4.1 Block 5: R Basics + Bioconductor . . . 21
4.2 Block 6: Differential Expression and ggplot2 . . . 26
4.2.1 Checkpoint: . . . 27
4.2.2 Resources . . . 27

5 Blocks 7–8: Statistics for Genomics (in R) 28
5.1 Block 7: Distributions, Tests, and Corrections . . . 28
5.2 Block 8: Models, Power, and Structure . . . 31
5.2.1 Can I move on? Checkpoint . . . 34
5.2.2 Resources . . . 34

6 Blocks 9–10: Applications — Single-Cell & GWAS 35
6.1 Block 9: Single-Cell RNA-seq, Start to Finish . . . 35
6.2 Block 10: GWAS Pipeline, Variant to Function . . . 37
6.2.1 Checkpoint: . . . 38
6.2.2 Resources . . . 38

7 Blocks 11–12: AI, Right Questions & Capstone 39
7.1 Block 11: Asking the Right Questions + AI Mastery . . . 39
7.1.1 The Question Map . . . 39
7.2 Block 12: Reproducible Pipeline + Capstone . . . 40
7.2.1 Capstone Project . . . 41

8 Running Project: From Growth Curves to QTL Mapping 42
8.1 Problem Set 1: Yeast Growth Curves . . . 42
8.1.1 Part A: Generate the Biomek Randomization CSV . . . 42

8.1.2 Part B: Fit Growth Curves & Extract Doubling Times . . . 43
8.1.3 Part C: Visualize & Test for Differences . . . 43
8.1.4 Part D: Permutation Test . . . 43
8.1.5 Part E: Power Analysis by Simulation . . . 43
8.2 Problem Set 2: Heritability & Mixed Models . . . 44
8.2.1 Part A: Read & Merge Data . . . 44
8.2.2 Part B: Visualize . . . 44
8.2.3 Part C: Fixed-Effects Linear Models . . . 45
8.2.4 Part D: Broad-Sense Heritability . . . 45
8.2.5 Part E: Mixed Model with Random Effects . . . 45
8.2.6 Part F: Normalized Response to Chemical X . . . 45
8.2.7 Part G: Heritability of the Normalized Trait . . . 46
8.3 Problem Set 3: Short-Read Alignment & Strain Identification . . . 46
8.3.1 Part A: Identify the Mystery Strain . . . 46
8.3.2 Part B: Visualize Meiotic Crossovers . . . 46
8.4 Problem Set 4: QTL Mapping . . . 47
8.4.1 Part A: Genome-Wide LOD Scan . . . 47
8.4.2 Part B: Permutation Threshold for Significance . . . 47
8.4.3 Part C: 1.5-LOD Drop Confidence Interval . . . 48
8.4.4 Part D: Candidate Genes & Biological Hypothesis . . . 48

9 Appendix A: Tools & Installation **49**
9.1 Core infrastructure (install once, before anything else) . . . 49
9.2 Command-line utilities not always installed by default . . . 49
9.3 Bioinformatics tools (install into a conda env) . . . 50
9.4 R and Python packages . . . 52

10 Appendix B: Programming Concepts Cheat Sheet **53**

11 Appendix C: Reading List (Prioritized) **54**
11.1 Start Here (the essentials) . . . 54
11.2 Go Deeper (when you're ready) . . . 54
11.3 Reference (keep on your shelf) . . . 54
11.4 Online Tools & Databases . . . 54
11.5 12-Block Overview . . . 55

How This Curriculum Works (Read This First)

The specific exercises are a vehicle for a broader set of skills. By the end of the curriculum, you should have:

- **General programming competency** in Unix, Python, and R: enough to parse files, write reusable scripts, manipulate tabular and genomic data, and make publication-quality figures.
- **A mental model of the software stack** that most bioinformatics work depends on: the OS and shell; conda environments and containers; language runtimes and their package ecosystems; domain-specific libraries and pipeline engines. Knowing which layer you are operating at is what lets you diagnose problems instead of flailing.
- **Competent use of AI tools** as part of everyday work: prompting with enough context and constraints to get runnable code, spotting hallucinated APIs, verifying against primary documentation, and treating AI output as a draft rather than an oracle.
- **The ability to ask the right question** before an analysis: what is the biological question, what would a positive result look like, what would a null result look like, what could fool you.
- **Project-structure instincts**: version-controlled code, reproducible environments, ignore-listed raw data, self-explanatory directory layout, journaled progress. Work that someone else (or future you) can clone and re-run.
- **Layer-by-layer evaluation skills**: at each step of a pipeline, the ability to say what should be true, check that it is, and recognize what has gone wrong when it isn't. This is where verification and real understanding live.

Traditional curricula front-load reading and back-load doing. This one inverts that order: every session is hands-on first, theory second.
Each session produces something tangible. You run a command, read the output, then read about why it behaves that way.
Jumping around is fine. Each sprint is self-contained enough that you can follow your curiosity without losing the thread.

The Three Rules

Rule 1: The 25-Minute Sprint. Every study session is broken into 25-minute sprints (Pomodoro). Each sprint has one clear goal and one tangible output. If you finish early, take the break early. If you are in flow, keep going, but never push past 50 minutes without a break.

Rule 2: Output Before Input. Do something before you read about it. Try the command first. Break something. Then read why it broke. You will remember mistakes and fixes much better than anything you passively read.

Rule 3: The Checkpoint. Every sprint ends with a concrete thing you made, produced, or proved works. Screenshot it, commit it, or write one sentence in your journal. This is your proof of progress.

How a Block Works

The curriculum is divided into 12 **blocks**. Each block is roughly 2 hours of focused work, organized as a small batch of 25-minute sprints with short breaks between them. Most blocks land around 3–5 sprints; check the chapter for the exact count. How you spread a block across the calendar is up to you: one 2-hour sitting, two 60-minute sittings, or a sprint a day all work. What matters is that you finish the block's sprints before moving on.

Sprint (25 min)	Hands-on: work through the sprint's goal and produce its checkpoint
Break (5 min)	Stand up, walk, look away from the screen
Next sprint	Continue, or stop here and pick up another day
End of block	Journal one sentence per sprint. Commit to Git.

Never force yourself past 50 minutes of sprinting without a real break. If you want to do more in a day, take 30+ minutes off before starting the next block.

Your Toolkit (Set Up in Block 0)

Before Block 1, spend one sitting setting this up. This is Sprint 0. **Appendix A is the complete install reference.** Every tool named in this curriculum (core infrastructure, command-line utilities, bioinformatics packages, R and Python libraries) is listed there with macOS and Linux install commands and official documentation links. Keep it open while you work through the curriculum. For a fast lookup of programming concepts (loops, pipes, recursion, regex, git verbs), see Appendix B; for a curated reading list and a block-by-block overview, see Appendix C.

On macOS, open `Terminal.app` (Cmd-Space, type "terminal"). On Linux, any terminal emulator works (GNOME Terminal, Konsole, xterm). On Windows, install WSL2 first (`https://learn.microsoft.com/windows/wsl/install`) and do everything inside the Ubuntu shell it gives you — the rest of this curriculum assumes a Unix-like shell. The `$` symbol at the start of a command is the shell prompt; do not type it.

Terminal	Everything runs here	Linux/macOS terminal; or WSL2 on Windows — see Appendix A
Miniforge	Package management without pain	`https://github.com/conda-forge/miniforge` — see Appendix A
VS Code	Editor with terminal built in	`https://code.visualstudio.com` (or your favorite editor)
Git + GitHub	Version control & your portfolio	`https://git-scm.com`; make a GitHub account now
Claude or ChatGPT	AI tutor always available	`https://claude.ai` (free tier works)
A journal.md file	One sentence per day = progress proof	Create it in your Git repo right now

Install everything. Open your terminal. Create a local Git repo called `bioinfo-journey` with a `journal.md` file inside it. You will push this repo to GitHub in Block 2, Sprint 8 once you have an SSH key set up; for now, just initialize it locally with `git init` and make your first commit.

Interleaving: Why This Curriculum Mixes Things Up

Research consistently shows that retention improves when topics are interleaved (mixed) rather than massed (all-Unix-then-all-Python).
Each block below has a primary focus, but deliberately connects to other skills. By Block 4 you are already using Python, Unix, and statistics together.
If you get stuck on one topic, switch to a different sprint in the same block. Coming back later with fresh eyes is not quitting; it is how interleaving works.

How to Use AI as Your Study Partner

AI tools are effective study aids. Treat them as a resource you can query any time for explanations, debugging, and code review.

- **Stuck on an error?** Paste the full error message into Claude or ChatGPT. Do not describe it; paste it.
- **Confused by a concept?** Ask: "Explain [concept] assuming I already know [thing I do know] but not [thing I don't]."
- **Want to check understanding?** Ask: "I think [my understanding]. Is that right? What am I missing?"
- **Need a break from reading?** Ask: "Give me a 5-minute hands-on exercise to practice [concept]."
- **Feeling overwhelmed?** Ask: "What is the single most important thing I should understand about [topic]?"

Always run AI-generated code before trusting it. AI tools hallucinate function names, invent packages, and mix up API versions. Test first, trust after.

Blocks 1–2: Unix Foundations

Primary: Command line, text processing, bash scripting | **Interleave:** Git, project setup

Most bioinformatics pipelines run on Unix, and most genomic file formats are plain text. If you can wrangle text in a terminal, you can wrangle genomes.

Every non-default tool named anywhere in this curriculum has install commands (macOS and Linux) and an official link in **Appendix A**. Flip to it the moment you hit a command that is not on your machine. Do not guess.

Block 1: The Command Line

Sprint 1: Navigate and Look Around *(25 min)*

- Open your terminal. Run each command and read its output before moving on: `pwd` (print working directory), `ls` (list), `ls -la` (long format, including hidden files), `cd ~` (go home), `cd /` (go to filesystem root), `cd -` (go to previous directory).
- Install `tree` if `tree -version` fails (Appendix A), then run `tree ~` with no arguments.
- Make a project directory: `mkdir -p ~/bioinfo/block1/{data,scripts,results}`. The curly braces are a *shell brace expansion* — the shell rewrites this into three `mkdir -p` arguments before running the command.
- Explore: what is in `/usr/bin`? How many entries? (`ls /usr/bin | wc -l`)

`Tab` completes filenames and commands. `Up-arrow` brings back the previous command. `Ctrl-C` cancels whatever is running. `Ctrl-L` (or `clear`) clears the screen. `man ls` opens the manual for a command; press `q` to quit. If you ever see a stray `>` prompt you did not expect, you left an open quote; press `Ctrl-C` and start over.

Run `tree ~/bioinfo` and paste its output into `journal.md` — you should see the three empty subdirectories you just made.

Sprint 2: Pipes: Chaining Small Tools *(25 min)*

Several commands below are too long to fit on one printed line, so either the source splits them across lines (with backslashes or pipes) or the PDF wraps them visually. Two rules that will save you hours:

- **A backslash (\) at the end of a line means "continue this command on the next line."** The backslash must be the *very last* character; a single trailing space turns `\ ` into "escape a space" and the line continuation is lost. Do not put anything (not even a `#` comment) after the backslash.
- **Inside a URL or any other token, \-then-newline is consumed with no replacement character.** So `https://host/\↩path` reassembles to `https://host/path` with no space inserted. This is why you will see URLs broken across lines below with no leading whitespace on the continuation.

A safer alternative when you have a choice: end the line on a pipe (`|`) or a logical operator (`&&`, `||`) instead. The shell knows a pipeline is incomplete, so it reads the next line automatically; no backslash is needed. If a copy-paste from the PDF misbehaves, suspect a smart-quote, a stripped backslash, or a soft line break — retype the break by hand.

- The pipe character `|` sends the standard output of one command into the standard input of the next. This is the core of the "small tools, composed" Unix design.
- Try: `echo 'ATGCGATCGATCG' | grep -o . | sort | uniq -c | sort -rn`. Read it left to right: print the string, split into one character per line, sort the lines, count adjacent duplicates, sort by count descending. You should see A, C, G, T each with a count.
- Download the S288C yeast reference genome (tarball is ∼4 MB). The URL is long, so the source below splits it with backslash continuation — the continuation lines start at column 1 with no leading spaces so the URL reassembles cleanly:

```
curl -O \
https://downloads.yeastgenome.org/sequence/\
S288C_reference/genome_releases/\
S288C_reference_genome_Current_Release.tgz
tar xzf S288C_reference_genome_Current_Release.tgz
cd S288C_reference_genome_*
```

- Find the genome FASTA: `ls *.fsa`. Look at the header lines with `grep '>' *.fsa | head`. Each header line starts with `>` and names one chromosome (I through XVI) plus the mitochondrial genome — that is what `grep '>'` pulls out.

Try `wget` instead of `curl -O`, or open https://www.yeastgenome.org/ and navigate to Sequence → Download. A long error with "Could not resolve host" usually means no network; "404 Not Found" means the file moved.

You chained four commands with pipes, counted nucleotide frequencies, and downloaded and un-tarred the yeast reference. You also inspected the FASTA and identified what each header line names.

Sprint 3: Grep and Awk: Extracting Lines and Columns *(25 min)*

- `grep` searches for patterns. `grep -c '>' file.fasta` counts sequences; `grep -v` inverts the match (keeps lines that do *not* match).
- `awk` extracts and transforms whitespace- or tab-separated columns. `awk -F'\t' '{print $1, $2,`

`$5}' file.vcf` prints columns 1, 2, and 5 of a tab-delimited file — for a VCF those are chromosome, position, and quality.

- Download a small yeast VCF to practice on. Same backslash-continuation pattern as the S288C download in Sprint 2:

  ```
  curl -O \
  https://ftp.ensembl.org/pub/current_variation/\
  vcf/saccharomyces_cerevisiae/\
  saccharomyces_cerevisiae.vcf.gz
  gunzip saccharomyces_cerevisiae.vcf.gz
  ```

  If the link has moved, browse https://ftp.ensembl.org/pub/current_variation/vcf/ and pick any small organism's VCF.
- Look at the file structure: `head -n 30 saccharomyces_cerevisiae.vcf`. Lines starting with `##` are metadata; the single line starting with `#CHROM` is the column header; all other lines are data records.
- Count variants per chromosome. Here the line endings are pipes, so the shell knows the pipeline is incomplete and no backslashes are needed:

  ```
  grep -v '^#' saccharomyces_cerevisiae.vcf |
    awk '{print $1}' |
    sort | uniq -c | sort -rn
  ```

  `grep -v '^#'` drops every header line; `awk` pulls out the chromosome column; `sort | uniq -c` tallies each chromosome; the final `sort -rn` orders by count.

Running the pipeline above produces a table with roughly 16 rows (one per yeast chromosome) with a count in front of each. Paste this table into `journal.md`.

Sprint 4: Put It All Together: Pick a Restriction Enzyme for Cloning

(25 min) A real scenario: you want to shotgun-clone roughly 3 kb fragments of the yeast genome into a plasmid. Three cheap 6-cutters are on the shelf: EcoRI (`GAATTC`), BamHI (`GGATCC`), and HindIII (`AAGCTT`). Before ordering any of them, predict from the genome sequence how often each enzyme cuts, and which one produces fragments closest to your 3 kb target. Do it all with pipes; no scripts yet. Paste every command and its output into `journal.md` as you go.

1. **Confirm and organize.** `head` the yeast FASTA — the first line should start with `>`. Then move the FASTA (and the VCF from Sprint 3) into `data/` in one pipeline: `ls *.fsa *.vcf 2>/dev/null | xargs -I{} mv {} ~/bioinfo/block1/data/`. Verify with `ls ~/bioinfo/block1/data/`.
2. **Measure the genome.** Strip headers and newlines, count characters:

   ```
   grep -v '>' ~/bioinfo/block1/data/*.fsa | \
     tr -d '\n' | wc -c
   ```

   For S288C, expect roughly 12 Mb. Save this number — you'll divide by it next.
3. **Count cut sites per enzyme.** Template for EcoRI:

   ```
   grep -v '>' ~/bioinfo/block1/data/*.fsa | \
     tr -d '\n' | grep -o GAATTC | wc -l
   ```

   Run it three times — once each for `GAATTC`, `GGATCC`, `AAGCTT`. Which cuts most? Fewest?
4. **Why one strand is enough.** All three recognition sites are palindromes (they read the same on both strands), so every real cut site is already captured by a forward-strand match. For non-palindromic motifs you would also have to scan the reverse complement; good to know before you generalize this approach.

5. **Compute average fragment size.** Average fragment size ≈ genome length ÷ number of cut sites. Use `bc`: `echo "12157105 / NSITES" | bc` (swap in your actual genome length and cut count). Expect numbers in the range of hundreds of bp to tens of kb.
6. **Pick your enzyme.** Which enzyme's average fragment size is closest to your 3 kb target? Write one sentence in `journal.md` explaining your choice, and flag which enzyme would be a bad idea (cuts so often you would get tiny fragments, or so rarely the fragments are too big to clone).

Your journal shows the `head` check, the move-into-`data/` pipeline, the genome length, cut-site counts for three enzymes, their average fragment sizes, and your enzyme pick with a one-sentence justification, all produced with pipes.

In Unix bioinformatics, the workflow is almost always: extract columns (`cut` / `awk`) → filter rows (`grep`) → sort → count (`uniq -c`). This pattern covers a large fraction of quick data exploration.

Block 2: Scripts, Environments & Git

Sprint 5: Your First Bash Script *(25 min)*

- Create the file: `nano ~/bioinfo/block1/scripts/count_variants.sh` (or use VS Code — any plain-text editor is fine).
- The first line must be the "shebang" `#!/usr/bin/env bash`, which tells the OS which interpreter to run. Follow it with `set -euo pipefail`: `-e` exits on the first error, `-u` errors on undefined variables, `-o pipefail` fails if any command in a pipeline fails.
- Write a script that reads a VCF filename from its first argument (`$1`) and prints a per-chromosome variant count. Template to fill in:

```
#!/usr/bin/env bash
set -euo pipefail

vcf="${1:?usage: count_variants.sh FILE.vcf}"
grep -v '^#' "$vcf" | awk '{print $1}' | \
  sort | uniq -c | sort -rn
```

  The `${1:?...}` syntax prints a usage message and exits if you forget to pass an argument.
- Make it executable and run it: `chmod +x count_variants.sh` then `./count_variants.sh ~/bioinfo/block1/data/saccharomyces_cerevisiae.vcf`.

The script prints the same per-chromosome counts you computed by hand in Sprint 3. Run it on a different VCF (same script, different data) and confirm it still works.

Sprint 6: Conda and Mamba: Isolated, Reproducible Environments *(25 min)*

- Create an environment: `mamba create -n biotools -c conda-forge -c bioconda samtools bcftools bedtools fastp`. The `-c` flags set channels; put `conda-forge` before `bioconda`.
- Activate it: `mamba activate biotools`. Verify: `samtools -version`.

- Export it: `mamba env export > environment.yml`. Anyone can then recreate your setup with `mamba env create -f environment.yml`.
- Deactivate: `mamba deactivate`. Run `samtools` again and confirm it is no longer on `PATH`. Environments isolate dependencies from each other and from the system.
- **Why mamba?** Conda's own dependency solver is slow on large environments. Mamba is a drop-in replacement (written in C++) that resolves the same specifications in seconds. Miniforge (https://github.com/conda-forge/miniforge) ships with it. You can still type `conda` everywhere; `mamba` is just faster.

- Forgetting `-c bioconda` when installing bioinformatics packages. You will get a `PackagesNotFoundError`.
- Installing into the `base` environment. Do not: `base` is for conda itself. Always make a named environment.
- Mixing `pip install` with `conda install` in the same environment without care. If you must, run `pip` last.
- Forgetting to `mamba activate`. A freshly opened terminal has no environment active; your `biotools` tools will look missing.

Conda packages are built per (OS, CPU) pair — `linux-64`, `osx-64`, `osx-arm64` (Apple Silicon), `linux-aarch64` (ARM Linux / some HPCs). Not every bioinformatics package has a build for every platform. On an M1/M2/M3 Mac, a missing `osx-arm64` build usually surfaces as `PackagesNotFoundError`; common fixes are to (1) create the env under Rosetta with `CONDA_SUBDIR=osx-64 conda create ...`, or (2) find an alternative package with native ARM support. An `environment.yml` built on one platform is not guaranteed to solve on another — expect small fix-ups when moving between laptop and cluster, and note the architecture in a comment at the top of the file.

Your `environment.yml` is a reproducible recipe. Commit it to Git, with a header comment noting which architecture built it.

Sprint 7: Containers: Reproducibility Beyond Conda *(25 min)*

A container is a single image file that bundles an entire Linux userland (OS libraries, the tool you care about, and everything that tool depends on) so that it runs identically on any compatible host. Think of the software stack as a stack of layers: the kernel at the bottom, system libraries on top of the kernel, language runtimes (Python, R) on top of those, and your scripts at the very top. Conda manages the *upper* layers: Python and R packages, plus a handful of binaries that link against them. Containers freeze *lower* layers too: the system libraries, `glibc` version, and compiled binaries that conda normally inherits from whatever host it runs on. A container is not a virtual machine (there is no guest kernel), so startup is near-instant and overhead is minimal.
Rule of thumb: use conda for interactive development (fast to edit, easy to iterate). Use a container when you ship a pipeline to a cluster, share an analysis with a collaborator on a different OS, or archive a workflow alongside a manuscript — situations where "works on my machine" has to become "works on yours."

- **Install Docker.** Docker is the developer-facing runtime. Install Docker Desktop on macOS (https://docs.docker.com/desktop/install/mac-install/) or Docker Engine on Linux (https://docs.docker.com/engine/install/). Verify with `docker -version`.
- **Pull a biocontainer.** The Biocontainers project auto-builds one container per bioconda package. Pull bcftools, which we'll use on the VCF you already have from Sprint 3:

```
docker pull \
  quay.io/biocontainers/bcftools:1.19--h8b25389_0
```

- **Run without installing.** Execute the container's `bcftools` with no other setup:

```
docker run --rm \
  quay.io/biocontainers/bcftools:1.19--h8b25389_0 \
  bcftools --version
```

  `-rm` discards the stopped container after it exits; the image stays cached for next time.
- **Mount your data.** Containers see an isolated filesystem by default. Map a host directory in so the tool can read local files. `cd` into the directory holding your VCF first, then:

```
cd ~/bioinfo/block1/data
docker run --rm -v "$PWD":/work -w /work \
  quay.io/biocontainers/bcftools:1.19--h8b25389_0 \
  bcftools view -h saccharomyces_cerevisiae.vcf | head -20
```

  `-v HOST:CONTAINER` mounts a host directory; `-w` sets the working directory inside. The container has no `bcftools` on the host, but it ran one anyway — and produced output from your real VCF.

Conda gives you speed and easy iteration at the cost of strict reproducibility: two users with the same `environment.yml` may still resolve to different package builds on different platforms. A container gives you a fixed binary that will run identically for years, at the cost of being heavier to build and harder to edit. Many published pipelines use both: conda during development, a container (often built from the conda env) for the frozen release.

You pulled a container, ran a bioinformatics tool from it without installing anything locally, and mounted a host directory into it so it could read your real VCF.

Sprint 8: Git and GitHub: Version Control and Your Portfolio *(25 min)*

- **Initialize and commit.**

```
cd ~/bioinfo && git init -b main && \
  git add -A && \
  git commit -m 'Blocks 1-2: Unix foundations'
```

- Edit something. `git diff` to inspect, `git add`/`git commit` to save, `git log -oneline` to review.
- **.gitignore.** Exclude anything big or generated: `*.bam`, `*.fastq.gz`, `*.vcf.gz`, `*.sam`. Never commit raw data. Commit the code that produces it and a pointer to where it lives.
- **Set up an SSH key for GitHub** (one-time; after this, no password prompts):

```
ssh-keygen -t ed25519 -C "you@example.com"
# press Enter to accept the default path;
# optionally set a passphrase
cat ~/.ssh/id_ed25519.pub
```

Copy the printed public key, then go to github.com → Settings → SSH and GPG keys → New SSH key and paste it. Test: `ssh -T git@github.com` should greet you by username.

- **Publish your repo.** Create an empty repo on github.com (don't add a README — your local copy already has files). Then from your local repo:

```
git remote add origin \
  git@github.com:<you>/bioinfo-journey.git
git push -u origin main
```

  The `-u` flag wires `main` to the remote so future `git pushes` need no arguments.
- **Clone and pull.** To download a repo onto a new machine, or to grab someone else's work: `git clone git@github.com:user/repo.git`. Inside an existing clone, `git pull` fetches new commits from the remote and merges them into your branch; that is how you keep a checkout in sync. Practice: clone your own repo into `/tmp/`, push a change from your main checkout, then run `git pull` in `/tmp/` and confirm the change arrived.

- `git push` refused with "non-fast-forward": someone (or past-you on a different machine) pushed commits you do not have. Run `git pull -rebase` first, then push.
- You committed a large data file by accident. Removing it with `git rm` only hides it going forward; the old commits still contain it. For a solo repo it is usually simplest to add the pattern to `.gitignore` and keep moving; for bigger accidents use `git filter-repo` or the BFG tool.
- `git status` shows files you did not edit. Line-ending conversion on Windows/WSL is a common culprit: set `git config -global core.autocrlf input`.

Your bioinfo-journey repo has real commits, ignores big files, pushes over SSH without password prompts, and you have proved you can clone and pull from it. This is the start of your portfolio.

Sprint 9: Integration Sprint: Reproducible Variant Summary

(50 min) The goal: take the work from Sprints 3–5 and turn it into a single reproducible script that runs inside the conda environment from Sprint 6, writes a timestamped report, and is tracked in the Git repo from Sprint 8. No new tools — just the ones you've already installed.

1. **Write `scripts/variant_report.sh`** that takes a VCF path as `$1` and:
   - starts with `#!/usr/bin/env bash` and `set -euo pipefail` (Sprint 5);
   - runs the per-chromosome variant tally from Sprint 3 (`grep -v '^#' | awk ...| sort | uniq -c | sort -rn`);
   - also runs `bcftools stats "$1" | head -30` from the `biotools` env (Sprint 6);
   - writes both outputs, prefixed by `date` and the input filename, into `results/report_$(date +%Y%m%d).txt`.
2. **Use the conda env.** Either `mamba activate biotools` before running the script, or have the script source conda and activate the env itself (`source "$(conda info -base)/etc/profile.d/conda.sh"; conda activate biotools`).
3. **Run it** on the yeast VCF from Sprint 3 and confirm the report file appears under `results/`.
4. **Commit the script and the report.** Per the `.gitignore` rules from Sprint 8, do not commit the raw VCF — but the script and the small text report are fair game.

A single command,

```
./scripts/variant_report.sh \
  ~/bioinfo/block1/data/saccharomyces_cerevisiae.vcf
```

produces a dated report under `results/` that you can hand to someone else along with your repo and your `environment.yml`, and they can reproduce it.

End of Blocks 1–2

Take a day to step away before starting Python. The command-line fundamentals are now in place.

Can I move on? Checkpoint:

- You can navigate any directory, find files, and chain 3+ commands with pipes
- You can write a bash script that processes a genomic file and logs output
- You have a conda environment with bioinformatics tools installed
- Your work is in a Git repo on GitHub

If not, spend 2–3 more days on the gaps. No rush.

Resources

- Software Carpentry: The Unix Shell (free, interactive)
- Buffalo – *Bioinformatics Data Skills*, Ch. 3, 7, 12
- explainshell.com — paste any command and see what each part does

Blocks 3–4: Python for Genomic Data

Primary: Data structures, Pandas, visualization | **Interleave:** Unix pipes, real genomic files

You just learned to process text files with grep and awk. Python does the same things but with reusable code, richer data structures, and the ability to make plots and run statistics in the same script.

You need Python 3 (not 2). Check with `python3 -version`; Miniforge's `base` environment already has a recent Python 3. Two ways to run code:

- **REPL / notebook.** Type `python3` at the shell for a live prompt, or `jupyter lab` after `mamba install -c conda-forge jupyterlab`. Good for exploration.
- **Script.** Put code in `file.py` and run `python3 file.py`. This is what the exercises below expect.

Indentation is syntax in Python: four spaces per level, consistent throughout a file. Mixing tabs and spaces produces `IndentationError`. Configure your editor to insert spaces when you press `Tab`.

Block 3: Core Python via Biology

Sprint 1: Data Structures for Biology *(25 min)*

Pythonic code is less about clever tricks and more about reaching for the right built-in collection. Each has a natural biological use case. **Open `python3` (or a Jupyter cell) and run each block before moving on** — read-only is not enough here.

- **Strings = DNA sequences.** Reverse-complement `ATGCGATCG` and verify by hand:

  ```
  seq = 'ATGCGATCG'
  comp = seq.translate(str.maketrans('ATCG','TAGC'))
  print(comp[::-1]) # reverse complement
  ```

  Do: now reverse-complement `AAACGTTTGCAGGGAATTC` and find the position of the EcoRI site (`GAATTC`) in both the original and the reverse complement using `str.find()`.
- **Lists = ordered, mutable collections.** Build a list of gene names and filter it:

  ```
  genes = ['BRCA1', 'BRCA2', 'TP53',
           'BRAF', 'MYC', 'BRD4']
  brc = [g for g in genes if g.startswith('BR')]
  print(brc) # 4 hits
  ```

  Do: change the comprehension to keep only genes whose name is exactly four characters long. Swap the outer brackets for parentheses — you now have a *generator expression*; pass it to `list()` to get the same result.
- **Tuples = immutable records.** Genomic coordinates are a natural fit:

  ```
  peak = ('chr1', 1000, 2000)
  chrom, start, end = peak # unpack
  ```

```
print(end - start) # 1000
```

Do: put three peak tuples into a `set()`; add a duplicate of the first; print `len()`. Because tuples are hashable, the duplicate is dropped — this is how you build "regions I've already processed."

- **Dictionaries = lookup tables.** Build a tiny codon table and translate three bases at a time:

```
codons = {'ATG': 'M', 'GCT': 'A',
          'AAA': 'K', 'TAA': '*'}
mrna = 'ATGGCTAAATAA'
prot = ''.join(codons[mrna[i:i+3]]
               for i in range(0, len(mrna), 3))
print(prot) # MAK*
```

Do: change one codon in `mrna` so the protein reads `MA*`.

- **Sets = membership and overlap.** Compare two "hit lists":

```
sampleA = {'BRCA1', 'TP53', 'MYC'}
sampleB = {'TP53', 'MYC', 'EGFR'}
print(sampleA & sampleB) # in both
print(sampleA - sampleB) # only A
```

Do: compute the union with `|` and confirm its size equals `len(A) + len(B) - len(A & B)`.

- **`collections.Counter` and `defaultdict`.** The right tool for "count things" and "group things":

```
from collections import Counter, defaultdict
seq = 'ATGCATGCATGCAA'
print(Counter(seq).most_common())

variants = [('chr1', 100), ('chr2', 50),
            ('chr1', 250), ('chr2', 80)]
by_chrom = defaultdict(list)
for chrom, pos in variants:
    by_chrom[chrom].append(pos)
print(dict(by_chrom))
```

Do: use `Counter` on a longer DNA string and call `.most_common(2)` to get the top two bases.

You reverse-complemented DNA, filtered a list, deduplicated peaks via a set, translated mRNA with a dict, intersected two sample hit lists, and grouped variants by chromosome with `defaultdict`. Idiomatic Python is largely about picking the right container rather than writing loops by hand.

Sprint 2: Parsing Genomic Files

(25 min)

A **parser** is just a function that turns a text file into structured data your program can use — a list of records, a dictionary, a dataclass. Every bioinformatics format (FASTA, FASTQ, VCF, BED, GFF) is plain text, but each has its own line-level rules: which lines are headers, which are data, what separates fields. Writing a parser is the act of teaching Python those rules. The recipe is always the same: open the file, walk through lines one at a time, recognize what each line means, and accumulate records. Today we'll build that recipe up step by step.

- **Step 1: read a file, line by line.** Make a tiny test file in your current directory called `toy.tsv`:

```
chr1 100 200
chr1 500 600
chr2 50 75
```

Then in Python:

```
with open('toy.tsv') as f:
    for line in f:
        print(repr(line))
```

The `with` block is a *context manager* — it closes the file automatically even if an exception is raised. `repr()` reveals the trailing `\n` on each line.

- **Step 2: split each line into fields.** Replace the `print` with:

```
chrom, start, end = line.strip().split('\t')
print(chrom, int(start), int(end))
```

`strip()` drops the trailing newline; `split('\t')` gives a list of three strings; `int()` converts the numeric fields. You just wrote a one-line BED parser.

- **Step 3: return structured records.** Wrap it in a function that returns a list of dictionaries:

```
def parse_bed(path):
    records = []
    with open(path) as f:
        for line in f:
            chrom, start, end = (
                line.strip().split('\t'))
            records.append({
                'chrom': chrom,
                'start': int(start),
                'end': int(end),
            })
    return records

print(parse_bed('toy.tsv'))
```

BED is tab-separated, 0-based, half-open — those are format conventions, not Python rules.

- **Step 4: now FASTA — one wrinkle.** A FASTA *record* is a header line starting with `>` followed by one or more sequence lines that you have to glue together. So unlike BED, you can't return a record from each line: you accumulate sequence lines until you see the next `>`. Write `toy.fa`:

```
>seq1 short
ATGCATGC
ATGC
>seq2 even shorter
GGGTTT
```

Then write `parse_fasta(path)` that returns a list of `(header, sequence)` tuples. Sketch:

```
def parse_fasta(path):
    records = []
    header, chunks = None, []
    with open(path) as f:
        for line in f:
            line = line.rstrip()
            if line.startswith('>'):
                if header is not None:
                    records.append(
                        (header, ''.join(chunks)))
                header = line[1:] # drop '>'
                chunks = []
            else:
                chunks.append(line)
        if header is not None: # last record
```

```
            records.append(
                (header, ''.join(chunks)))
    return records
```

Run `parse_fasta('toy.fa')` and confirm `seq1`'s sequence is `ATGCATGCATGC` (the two lines joined).

- **Step 5: handle malformed input.** Add a deliberately broken line to `toy.tsv` (e.g. remove a tab) and rerun `parse_bed()`. You'll get a `ValueError`. Wrap the inner work in `try/except ValueError`: print a warning naming the line, then `continue`. A robust parser tolerates one bad row without crashing the whole run.

You wrote two parsers from scratch: a single-line BED parser and a multi-line FASTA parser. Both run on tiny test files you wrote by hand. Your BED parser also prints a warning and keeps going when you feed it a malformed line.

Sprint 3: Generators: Streaming Files Larger Than RAM *(25 min)*

- A generator function uses `yield` instead of `return`; it produces items one at a time without building the full list in memory.
- Convert your FASTA parser to a generator. Usage changes from `records = parse(path)` followed by `for r in records` to a single `for r in parse(path)`. The file stays open only as long as the loop runs.
- Why this matters: a whole-genome VCF or a raw FASTQ can be tens of GB. Loading all of it into memory will swap your machine or crash the job.

A generator is consumed once. After `for r in parse(path): ...`, the generator is exhausted; running the loop again yields nothing. If you need the data twice, either call `parse(path)` again (re-opens the file) or materialize with `list(parse(path))`.

Your FASTA parser yields one record at a time and runs on a 1 MB test file without loading all of it into memory. Verify with `import sys; sys.getsizeof(parse(path))` before and after converting.

Sprint 4: Pythonic Idioms and Type Hints *(25 min)*

Writing Python that other people (and future you) can read quickly is mostly a matter of using the idioms the language was built around. Each of these takes a few minutes to learn and saves hours of debugging.

- **f-strings.** `f"{gene} on chr{chrom} (padj={pval:.2e})"` — variables, formatting, and expressions in one readable line. Use these exclusively for user-facing strings; they're faster than `%` or `.format()` and much harder to misread.
- **`pathlib` for filesystem paths.**

  ```
  from pathlib import Path
  data = Path.home() / "bioinfo" / "block1" / "data"
  for vcf in data.glob("*.vcf"):
      print(vcf.stem, vcf.stat().st_size)
  ```

  Path objects beat string concatenation: they handle OS differences, support `/` for joining, and expose `.stem`, `.suffix`, `.parent`, `.exists()`.

- **`enumerate` and `zip`.** Almost never write `for i in range(len(xs))` — use `for i, x in enumerate(xs)` or `for a, b in zip(xs, ys)`. These are the canonical ways to walk sequences.
- **Dataclasses for records.**

  ```
  from dataclasses import dataclass

  @dataclass
  class Variant:
      chrom: str
      pos: int
      ref: str
      alt: str
      qual: float = 0.0
  ```

  You get `__init__`, `__repr__`, and equality for free, with attribute access (`v.pos`) instead of `v["pos"]`. Prefer these over parallel lists or dicts-of-fields for anything with a fixed shape.
- **Type hints.** Annotations like `def parse_vcf(path: Path) -> list[Variant]:` are not enforced at runtime, but tools (`mypy`, Pyright, your editor) use them to catch bugs before you run the code, and they're the best function documentation you can write. Start with hints on function signatures and dataclass fields — don't try to annotate everything at once.
- **Context managers beyond `open()`.** `with pysam.AlignmentFile(...) as bam:`, `with tempfile.TemporaryDirectory() as d:`, `with contextlib.suppress(FileNotFoundError):`. Any resource that needs deterministic cleanup — file handles, temp dirs, database cursors, locks — is a candidate.

Before adding a dependency, check `collections`, `itertools`, `functools`, `pathlib`, `dataclasses`, `contextlib`, `statistics`. The Python standard library solves most problems you'll hit in genomics scripting, and stdlib code ages well — no surprise breaking changes when your analysis reviewer tries to rerun it two years later.

You wrote a function with a type-hinted signature, a dataclass for one biological record, and a `pathlib`-based file iteration. This is Python that will read well in a pull request.

You already know `awk '{print $1}'` extracts column 1. The Python equivalent is `line.split('\t')[0]`. Your Unix skills transfer directly. When in doubt, prototype in awk, then scale up in Python once the logic has more steps.

Block 4: Pandas, Plotting, and Bio Libraries

Sprint 5: Pandas for Tabular Genomic Data *(25 min)*

- Load GWAS summary statistics: `df = pd.read_csv('sumstats.tsv', sep='\t', dtype={'CHR': str})`. Explicit `dtype` for chromosome prevents Pandas from silently parsing "X" as NaN.
- Filter: `sig = df[df['P'] < 5e-8]`, the genome-wide significant hits.
- Group by: `df.groupby('CHR')['P'].min()`, the best p-value per chromosome.
- Merge: `pd.merge(genotypes, phenotypes, on='SAMPLE_ID')`, the most common GWAS join.

Download one from the GWAS Catalog (https://www.ebi.ac.uk/gwas/) or the PGS Catalog (https://www.pgscatalog.org). If you just want to practice the mechanics, fabricate a minimal file with `numpy.random` and a few thousand rows; the commands above work identically.

You loaded, filtered, and grouped a summary-statistics table. Compare `df.shape` before and after filtering to confirm the filter did something.

Sprint 6: Visualization *(25 min)*

- Manhattan plot: scatter of $-\log_{10}(P)$ vs. genomic position, colored by chromosome.
- QQ plot: expected vs. observed $-\log_{10}(P)$; the standard GWAS diagnostic for inflation.
- Use matplotlib for control and seaborn for defaults. Always label axes and include a title.
- Save with `plt.savefig('manhattan.png', dpi=150, bbox_inches='tight')`.

You produced a Manhattan plot and a QQ plot as PNGs. Commit both to your repo.

Sprint 7: NumPy and SciPy for Fast Math *(25 min)*

- NumPy arrays are typically two orders of magnitude faster than Python lists for numeric math. Use `np.mean(quality_scores)` rather than `sum(xs)/len(xs)`.
- Vectorized operations avoid explicit loops: `genotypes * 2` multiplies every element at once.
- SciPy provides standard hypothesis tests: `scipy.stats.ttest_ind(group1, group2)`.
- Multiple testing correction: `from statsmodels.stats.multitest import multipletests`.

You ran a t-test in Python, then applied Benjamini–Hochberg correction to a vector of p-values and compared the number of significant hits before and after.

Sprint 8: Bio Libraries: pysam, Biopython, scanpy *(25 min)*

- pysam: read BAM/CRAM files directly. Count reads mapping to a gene interval in a few lines.
- Biopython: `SeqIO.parse()` handles FASTA, FASTQ, and GenBank; `Entrez.efetch()` queries NCBI.
- scanpy: the single-cell analysis framework used heavily in Blocks 9–10.
- scikit-allel: population genetics in Python (allele frequencies, LD, PCA from VCF data).

You queried NCBI from Python (e.g., retrieved a GenBank record by accession) and counted reads in a BAM file for one interval.

End of Blocks 3–4

You just picked up a new language and applied it to biology. Take a day off. Let it consolidate.

Checkpoint:

- You can parse any common genomic file in Python (FASTA, BED, VCF, TSV)
- You can use Pandas to load, filter, group, and merge tabular data
- You can make a Manhattan plot and a QQ plot
- You understand why vectorized NumPy operations are faster than Python loops

Resources

- McKinney – *Python for Data Analysis* (the Pandas bible)
- VanderPlas – *Python Data Science Handbook* (free online)
- Real Python (realpython.com) — searchable tutorials for anything

Blocks 5–6: R, Bioconductor & Differential Expression

Primary: Bioconductor, differential expression, ggplot2 | **Interleave:** Statistics concepts, Unix file prep

R has Bioconductor, a collection of thousands of biology-specific packages with no direct Python equivalent. DESeq2, edgeR, GenomicRanges, and Seurat all live there. For working genomicists, R and Python are complements rather than substitutes.

R is not installed by default on macOS or most Linux distributions. Download the pre-built binary for your OS/architecture from CRAN (https://cran.r-project.org), then install RStudio Desktop for a comfortable IDE (https://posit.co/download/rstudio-desktop/). See Appendix A for more.

Sprints 2–5 read four small data files from the `data/` subdirectory of this curriculum's repo: `counts.tsv`, `sample_info.tsv`, `de_results.csv`, and `peaks.bed`. Either clone the repo and `setwd()` into it, or copy the files into the working directory of your R session. `file.exists("data/counts.tsv")` should return `TRUE` before you start Sprint 2.

Block 5: R Basics + Bioconductor

Sprint 1: R in 25 Minutes: Vectors, Lists, Data Frames *(25 min)*

- **Vectors.** `x <- c(1, 2, 3); mean(x)`. Everything in R is a vector; even a single number is a length-1 vector. Vectors are homogeneous: one type throughout (numeric, character, or logical).
- **Lists.** Heterogeneous containers:

  ```
  res <- list(pval = 0.003,
              estimate = 1.42,
              samples = c("A", "B", "C"))
  res$pval # 0.003
  res[["samples"]] # c("A", "B", "C")
  ```

  Access elements by name with `$` or by name/position with `[[`. Every complex R object (`lm()` output, a `DESeqDataSet`, a phylo tree) is a list underneath. Learn `str(obj)` now; you will use it constantly to inspect unfamiliar objects.
- **Data frames.** A list of equal-length columns with row-and-column indexing: `df <- read.csv('samples.tsv', sep='\t')`. First three commands on any new data frame: `head(df)`, `names(df)`, `str(df)`.

- **Indexing gotchas.** R is 1-based: `df[1,]` is row 1, not row 2. Negative indices exclude: `df[-1,]` drops row 1. Logical vectors filter: `df[df$pval < 0.05, ]`.

Assignment operator: `<-` vs. `=`. The R convention is `<-` for creating variables (`x <- 5`). The `=` sign is also accepted at the top level and does the same thing, but inside a function call it does something *different*: it names an argument rather than creating a variable. Consider:

```
mean(x = c(1, 2, NA), na.rm = TRUE) # names the first and
                                    # second arguments
mean(x <- c(1, 2, NA), na.rm = TRUE) # creates a variable x
                                    # in the surrounding
                                    # scope AND passes it
                                    # positionally
```

The first form is what you want almost every time. The second form has a side effect (a new `x` visible outside the call) which is almost never what you want and is a classic source of confusing bugs. So: use `=` to match arguments inside `f(arg = value)` calls, and use `<-` to create variables at the top level. In practice, if you follow that rule both sides of your code read correctly and you never have to think about it again.
Does it really matter? Mostly for readability and for avoiding the side-effect bug above — execution-wise, `x <- 5` and `x = 5` at the top level produce the same bindings. Most R style guides (tidyverse, Bioconductor, Google's) specify `<-` for assignment, so code you read in the wild will look that way. Editors like RStudio bind `Alt-` (Windows/Linux) or `Option-` (macOS) to type `<-` in one keystroke.
`NA` means "missing" and *propagates* through arithmetic: `mean(c(1,2,NA))` returns `NA` unless you pass `na.rm = TRUE`. **Factors** look like strings but behave like integers; if a column's contents surprise you, run `class()` first.

You loaded a TSV, inspected its structure with `str()`, pulled one column and one row, and can describe the difference between a vector, a list, and a data frame. Apply functions and the tidyverse come next.

Sprint 2: Lists, the Apply Family, and Functional R *(25 min)*

R grew up as a functional language: instead of writing for-loops, you hand a function to another function that runs it over a collection. This pattern is everywhere in R code, so you need to read it fluently even if you prefer to write loops yourself.

Load the count table for this sprint first. `counts.tsv` has 50 genes and four count columns (`sample1`–`sample4`):

```
counts_df <- read.table("data/counts.tsv",
                        header = TRUE, sep = "\t")
head(counts_df)
counts_mat <- as.matrix(
  counts_df[, c("sample1", "sample2",
               "sample3", "sample4")])
rownames(counts_mat) <- counts_df$gene_id
```

Now you have `counts_mat`: 50 rows (genes) by 4 columns (samples), with gene names as row names.

- **`apply()` — walk rows or columns of a matrix.** First arg = matrix; `1` = rows, `2` = columns; third arg = the function.

  ```
  row_means <- apply(counts_mat, 1, mean)
  ```

```
col_sds <- apply(counts_mat, 2, sd)
head(sort(row_means, decreasing = TRUE))
col_sds
```

`row_means` is a per-gene mean count; `col_sds` is a per-sample standard deviation.

- **`lapply()` — run f over a list, return a list.** Treat the columns of `counts_mat` as a list of vectors:

```
per_sample <- as.list(as.data.frame(counts_mat))
means <- lapply(per_sample, mean)
means$sample1
```

- **`sapply()`** does the same and simplifies to a vector/matrix when it can: `sapply(per_sample, mean)` returns a named numeric vector. Convenient interactively; prefer `vapply()` in scripts — it type-checks the return shape and fails loudly when a single element deviates.
- **`tapply()` — split by a grouping vector, then apply.** Use the `chromosome` column to compute mean expression per chromosome for sample1:

```
tapply(counts_df$sample1,
       counts_df$chromosome, mean)
```

- **`mapply()` / `Map()` — over multiple parallel inputs.** `mapply(t.test, group_a_list, group_b_list)` runs a t-test pairing element 1 of each, element 2 of each, etc.
- **Why this matters.** DESeq2 results come back as a list; `apply()` is how you compute per-gene statistics; `lapply()` appears in every production R pipeline. Loops are not wrong, but apply-family code is shorter and more idiomatic.
- **The modern equivalent.** The `purrr` package (part of the tidyverse) gives you `map()`, `map_dbl()`, `map_chr()`, `map2()`, `pmap()` — same idea, stricter return types. We'll use `purrr::map_dbl()` in the next sprint.

You loaded `counts.tsv`, computed per-gene means and per-sample standard deviations, and got mean expression per chromosome with one `tapply` call — without writing a single explicit loop. This is the R way.

Sprint 3: Tidyverse: A Modern Layer on Top of R *(25 min)*

Base R works, but routine data wrangling is verbose. The tidyverse is a family of packages (`dplyr`, `tidyr`, `readr`, `purrr`, `ggplot2`, `stringr`, `forcats`, and more) that share a consistent grammar: pipe a data frame through a sequence of *verbs*, each returning a modified data frame. Once you internalize the verbs, most analyses become one-glance readable.

Install once with `install.packages("tidyverse")` in an R session — that single call pulls in every package listed above. Docs: https://www.tidyverse.org.

- **The pipe. `%>%`** (from `magrittr`, re-exported by dplyr) and the newer base-R `|>` (R $\geq$ 4.1) both do the same thing: pass the left-hand side as the first argument of the right-hand side. They're Unix `|` for data frames.
- **Load the example DE table.** `de_results.csv` has columns `gene`, `chromosome`, `sample`, `log2FC`, `p.value`, and `pAdj`. The dot in `p.value` is just part of the column name — back-tick it when dplyr needs to quote it.

```
library(readr)
library(dplyr)
de_results <- read_csv("data/de_results.csv")
glimpse(de_results)
```

- **The five verbs you'll use constantly.**

```
de_results |>
  filter(pAdj < 0.05) |>
  mutate(direction = if_else(log2FC > 0,
                          "up", "down")) |>
  group_by(chromosome, direction) |>
  summarise(n = n(), .groups = "drop") |>
  arrange(desc(n))
```

  `filter` picks rows; `mutate` adds/changes columns; `group_by` + `summarise` compute per-group statistics; `arrange` sorts; `select` picks columns. Five verbs cover 80% of real analysis.
- **Tibbles.** `tibble()` creates a modern data frame that prints neatly (no auto-factor conversion, no surprising type coercion). `readr::read_csv()` returns a tibble; it behaves like a `data.frame` everywhere else.
- **Reshaping.** `pivot_longer()` and `pivot_wider()` (`tidyr`) go between wide and long formats — critical for plotting in ggplot2 and for joining disparate datasets.
- **Joins.** `inner_join()`, `left_join()`, `anti_join()`. Merge GWAS summary stats with gene annotations by SNP ID in one line.
- **`across()` for column-wise operations.** `mutate(across(starts_with("sample_"), log1p))` log-transforms every sample column at once.

Tidyverse is great for interactive analysis and scripts humans will read. Package authors often prefer base R internally — it has zero dependencies and a more stable API. Both styles are valid; most real projects mix them.

You chained five dplyr verbs to go from raw DE results to a ranked table of up/down counts per chromosome. Compare the equivalent base-R code (`aggregate`, nested `[`) — same output, very different ergonomics.

Sprint 4: Bioconductor: GRanges for Genomic Intervals *(25 min)*

- Install (one-time):

```
install.packages("BiocManager")
BiocManager::install(c("GenomicRanges",
                       "rtracklayer"))
```

- **Create a GRanges by hand.**

```
library(GenomicRanges)
gr <- GRanges(seqnames = "chrI",
              ranges = IRanges(start = 100,
                               end = 200))
gr
```

- **Load real intervals from a BED file.** `peaks.bed` is the BED5 file (chrom/start/end/name/score) of 30 ChIP-style peaks on yeast chromosomes:

```
library(rtracklayer)
peaks <- import("data/peaks.bed")
peaks # a GRanges with 30 ranges
seqnames(peaks)
width(peaks)
```

`rtracklayer::import()` reads BED, GFF, GTF, and BigWig with the same call.

- **Overlap two GRanges.** Make a few fake gene intervals and find which peaks fall in genes:

```
genes <- GRanges(
  seqnames = c("chrI", "chrII", "chrIV"),
  ranges = IRanges(start = c(1, 100000, 800000),
                   end = c(60000, 200000, 900000)))
hits <- findOverlaps(peaks, genes)
hits
```

`findOverlaps()` is the in-R counterpart of `bedtools intersect`.

> You imported a real BED file as a `GRanges` object and ran an overlap against a hand-built set of gene intervals. Same concept as bedtools, different syntax.

Sprint 5: SummarizedExperiment: The Standard Bioconductor Container *(25 min)*

- **Build one from the files you already have.** A `SummarizedExperiment` bundles three things into a single object: a counts matrix, gene-level metadata (`rowData`), and sample-level metadata (`colData`). Construct it from `counts.tsv` and `sample_info.tsv`:

```
library(SummarizedExperiment)

counts_df <- read.table("data/counts.tsv",
                        header = TRUE, sep = "\t")
sample_df <- read.table("data/sample_info.tsv",
                        header = TRUE, sep = "\t")

count_mat <- as.matrix(
  counts_df[, c("sample1", "sample2",
                "sample3", "sample4")])
rownames(count_mat) <- counts_df$gene_id

se <- SummarizedExperiment(
  assays = list(counts = count_mat),
  rowData = DataFrame(chromosome = counts_df$chromosome),
  colData = DataFrame(sample_df,
                      row.names = sample_df$sample))
se
```

- **Explore the object you just built.**

```
dim(se) # genes x samples
assay(se)[1:5, 1:4] # top-left of count matrix
rowData(se) # per-gene metadata
colData(se) # per-sample metadata
```

- **Why this matters.** Most Bioconductor analysis tools (DESeq2, edgeR, limma) accept a `SummarizedExperiment` directly. Bundling assay + rowData + colData in one object means you can subset by sample (`se[, se$condition == "treated"]`) without ever desynchronizing the count matrix from its metadata — a classic source of bugs when you keep them in parallel objects.
- **Optional: pull a public dataset.** For a larger real-world example, try `recount3` (pre-aligned RNA-seq from thousands of public studies) or `GEOquery` (any GEO series); both return `SummarizedExperiment` objects with the same accessor functions.

You built a `SummarizedExperiment` from `counts.tsv` + `sample_info.tsv`, and `dim(se)`, `colData(se)`, and `rowData(se)` all return something sensible. This is the object DESeq2 expects in the next sprint.

Block 6: Differential Expression and ggplot2

Sprint 6: DESeq2: Standard RNA-seq Differential Expression *(50 min)*

- Build a `DESeqDataSet` from your `SummarizedExperiment` plus a design formula (e.g., `~condition`).
- Run `dds <- DESeq(dds); res <- results(dds)`. Two function calls do the whole analysis.
- Understand what happened under the hood: normalization (size factors), dispersion estimation with empirical Bayes shrinkage, a negative-binomial GLM fit per gene, and a Wald test for each contrast.
- Extract results: `res[order(res$padj), ]` gives genes sorted by adjusted p-value. Save the table to disk — you'll plot it with ggplot2 in Sprint 7.

You ran a differential expression analysis end-to-end. Sanity checks: a mostly-uniform p-value histogram away from zero, and a few genes with very low `padj`.

Sprint 7: ggplot2: Grammar of Graphics *(25 min)*

- The basic form: `ggplot(data, aes(x, y)) + geom_point() + theme_minimal()`.
- Layer more geoms: `+ geom_violin() + geom_boxplot(width = 0.1)`.
- Facets: `+ facet_wrap(~chromosome)` splits into small multiples by a variable.
- Save with `ggsave('figure1.pdf', width = 8, height = 6)`.
- **Volcano plot.** Use the DE results from Sprint 6 (or `de_results.csv` from Sprint 3) to make a volcano plot of $-\log_{10}(p)$ vs. $\log_2$(fold change):

```
library(ggplot2)
library(readr)
de <- read_csv("data/de_results.csv")
ggplot(de, aes(x = log2FC,
               y = -log10(p.value))) +
  geom_point(aes(color = pAdj < 0.05),
             alpha = 0.7) +
  scale_color_manual(values = c("grey60",
                                "firebrick")) +
  labs(x = "log2 fold change",
       y = "-log10(p-value)",
       color = "FDR < 0.05") +
  theme_minimal()
ggsave("volcano.pdf", width = 6, height = 5)
```

A well-behaved dataset shows a roughly symmetric volcano with the most extreme points highlighted.

You produced a multi-panel figure with labeled axes, a title, and a saved PDF in your repo, plus a volcano plot from your DE results.

Sprint 8: Compare DE Methods *(25 min)*

- Run the same dataset through `limma-voom`: `voom(counts)` → `lmFit` → `eBayes` → `topTable`.
- Compare the result tables: how many genes are significant at FDR 0.05 in DESeq2 but not limma? Vice versa?
- Visualize the overlap with a Venn diagram or UpSet plot.
- Consider why they differ: DESeq2 fits a negative-binomial GLM; limma-voom transforms counts to approximately normal and fits a linear model with precision weights. Which assumption is more plausible for your data?

You compared two methods on the same data, quantified the overlap, and articulated one reason the results might disagree.

End of Blocks 5–6

You now have working fluency in bash, Python, and R, and can run a differential expression analysis end-to-end. Take a break before moving on.

Checkpoint:

- You can manipulate data in R with tidyverse
- You understand GRanges and SummarizedExperiment
- You can run a DESeq2 analysis end-to-end
- You can make publication-quality ggplot2 figures
- You can explain what empirical Bayes shrinkage does and why DESeq2 uses it

Resources

- R for Data Science, 2nd Ed (r4ds.hadley.nz) — free online, excellent
- DESeq2 vignette (follow the Beginner's Guide first, then the full vignette)
- StatQuest YouTube: the DESeq2 and edgeR episodes are outstanding
- Bioconductor workflows: bioconductor.org/packages/release/workflows/

Blocks 7–8: Statistics for Genomics (in R)

Primary: Distributions, hypothesis tests, multiple testing, linear models, power, PCA, Bayesian ideas | **Language:** R throughout

Many weak genomics papers have a statistics problem, not a coding problem.
You do not need to derive proofs. You do need to know: which distribution fits my data, what test is appropriate, what happens when my assumptions are wrong, and how large a sample do I need before starting.
This block teaches by simulation in R. You generate data you already know the truth about, break assumptions on purpose, and observe what the methods do. R has the most direct vocabulary for this kind of work: one function per distribution (`rnorm`, `rbinom`, `rpois`, `rnbinom`), one-line model fitting (`lm`, `glm`), first-class support for resampling (`sample`, `replicate`), and the standard p-value adjustment (`p.adjust`).

Open RStudio (or another R IDE). Create a fresh script for each sprint and save it under `~/bioinfo/block7/scripts/`. Install `MASS`, `broom`, `ggplot2`, `lme4`, and (via BiocManager) `qvalue` and `limma` up front:

```
install.packages(c("MASS", "broom", "ggplot2", "lme4"))
BiocManager::install(c("qvalue", "limma"))
```

Every sprint below calls `set.seed(1)` at the top so your simulations match the answer key.

Block 7: Distributions, Tests, and Corrections

Sprint 1: Which Distribution Fits My Data? *(25 min)*

- Simulate 10,000 draws from each of four common distributions and plot histograms on one page:

  ```
  set.seed(1)
  par(mfrow = c(2, 2))
  hist(rnorm(1e4, mean = 5, sd = 2),
       breaks = 50, main = "Normal")
  hist(rbinom(1e4, size = 100, prob = 0.1),
       breaks = 50, main = "Binomial")
  hist(rpois(1e4, lambda = 2),
       breaks = 50, main = "Poisson")
  hist(rnbinom(1e4, size = 1, mu = 2),
       breaks = 50, main = "Negative Binomial")
  ```

  Compare shapes. Poisson and NB have the same mean; NB has a heavier right tail.
- Overlay a real dataset. Load a counts vector from Block 5 (`counts <- assay(se)[, 1]`) and plot its histogram. Raw RNA-seq counts are not normal — they are overdispersed and much better fit by a negative binomial.

- Fit an NB to the data and read off the parameters:

  ```
  library(MASS)
  fit <- fitdistr(counts, "negative binomial")
  fit$estimate # size (dispersion), mu (mean)
  ```

  Small `size` means high overdispersion (long tail). This is why DESeq2 and edgeR use NB models rather than Poisson or normal.

R names each distribution with four prefixes: `d`ensity, `p`robability (CDF), `q`uantile, `r`andom draws. So `rnorm` draws random normals, `pnorm` gives $P(X \leq x)$, `qnorm` gives the inverse CDF, `dnorm` the density. The same four-prefix pattern covers `binom`, `pois`, `nbinom`, `beta`, `hyper`, `unif`, `t`, `chisq`, `f`, and more.

- Binomial: counting successes (alt allele reads out of total reads at a site)
- Poisson: rare events with a known rate (mutations per gene per generation)
- Negative binomial: counts with extra variation (RNA-seq — the big one)
- Normal: continuous measurements, often after transformation (log-expression, height)
- Beta: values between 0 and 1 (allele frequencies, methylation β-values)
- Hypergeometric: overlap significance (gene-set enrichment, Fisher's exact test)

You produced a four-panel histogram comparing simulated distributions, fit an NB to a real counts vector, and reported its mean and dispersion. Save the figure and the fit summary in your repo.

Sprint 2: Hypothesis Tests and Permutations *(25 min)*

R ships every common test as a one-liner. The more useful skill is choosing the right one and checking its assumptions.

- **Two-sample t-test.** `t.test(x, y)` tests equality of means; Welch's correction is the default (no equal-variance assumption). Inspect: the returned object is a list with `statistic`, `p.value`, `estimate`, `conf.int`. Access them with `$` or cleanly with `broom::tidy(t.test(x, y))`.
- **Wilcoxon / Mann–Whitney.** `wilcox.test(x, y)` is the non-parametric alternative — ranks instead of values. Use when distributions are clearly non-normal or when there are outliers.
- **Counts: Fisher's exact.** `fisher.test(matrix(c(a, b, c, d), 2, 2))` on a 2×2 table; used everywhere for gene-set enrichment (significant genes $\times$ pathway membership).
- **Permutation test (when in doubt).** Any test-statistic, any null, one loop:

  ```
  set.seed(1)
  obs <- t.test(x, y)$statistic
  combined <- c(x, y); n_x <- length(x)
  null <- replicate(1000, {
    perm <- sample(combined)
    t.test(perm[1:n_x],
           perm[(n_x + 1):length(combined)])$statistic
  })
  p_perm <- mean(abs(null) >= abs(obs))
  ```

  Assumption: exchangeability of group labels under the null. No normality assumption. Useful whenever the parametric test is doubtful.
- **Effect size, not just p-value.** A significant result with a tiny effect is usually not interesting. Report `mean(x) - mean(y)` alongside Cohen's $d = (\mu_1 - \mu_2)/s_{\text{pooled}}$, and a 95% confidence interval (from `t.test` directly).

On a simulated two-group dataset (`x <- rnorm(30, 0, 1); y <- rnorm(30, 0.5, 1)`), run `t.test`, `wilcox.test`, and your permutation test. The three p-values should agree to within rounding. Record all three in your journal, with effect size and 95% CI.

Sprint 3: Multiple Testing in R

(25 min)

- **One million null tests.** Simulate the genomics-scale problem:

```
set.seed(1)
p <- runif(1e6)
sum(p < 0.05) # expect ~50,000: pure noise
```

- **`p.adjust` is the one function to remember.** `p.adjust(p, method = "bonferroni")` divides the threshold by m; very conservative. `p.adjust(p, method = "BH")` is Benjamini–Hochberg and controls the false discovery rate (expected proportion of false calls *among rejections*). The FDR approach is the genomics standard.

```
sum(p.adjust(p, "bonferroni") < 0.05) # should be 0
sum(p.adjust(p, "BH") < 0.05) # should be 0
```

- **Add real signal.** Now mix a little: 99% null, 1% from a shifted alternative.

```
set.seed(2)
m <- 1e5; m1 <- 1e3
p_null <- runif(m - m1)
# alternatives: small p-values
p_alt <- rbeta(m1, 0.2, 1)
p_mix <- c(p_null, p_alt)
sum(p.adjust(p_mix, "BH") < 0.05)
```

  You should recover a substantial fraction of the 1000 true positives with BH while controlling expected FDR at 5%.

- **QQ plot with ggplot2.**

```
library(ggplot2)
df <- data.frame(
  obs = -log10(sort(p_mix)),
  exp = -log10(ppoints(length(p_mix))))
ggplot(df, aes(exp, obs)) +
  geom_point(size = 0.3) +
  geom_abline(slope = 1, intercept = 0,
              color = "red") +
  labs(x = expression(-log[10]("expected P")),
       y = expression(-log[10]("observed P")))
```

  Points on the diagonal are null; upward deviation at the tail is signal. Deviation across the whole line (not just the tail) is inflation — investigate.

- **Bonus: `qvalue` for π_0 estimation.** The `qvalue` package estimates π_0, the proportion of truly null tests, and gives per-test q-values (per-test FDR). `qvalue(p_mix)$pi0` should be near 0.99 on the simulation above.

- Treating BH FDR as the probability *this specific* result is false. It is the expected proportion of false calls among all rejections.
- Reporting an uncorrected p-value alongside 10,000 tests without saying which correction was

used.
- Using Bonferroni after BH (or vice versa) on the same p-vector: pick one.
- Applying FDR to a p-value vector that was produced under a broken null (e.g., inflation from unmodeled structure). FDR control assumes the p-value calibration is correct to begin with.

You simulated a million null p-values and a mixture of nulls plus real signal, applied Bonferroni and BH, produced a QQ plot that shows tail deviation only for the mixture, and noted π_0 from `qvalue`. Report the three numbers (Bonferroni rejections, BH rejections, π_0) in your journal.

Block 8: Models, Power, and Structure

Sprint 4: Linear Models, GLMs, and Diagnostics

(25 min) A GWAS fits a linear model per variant. Differential expression fits a GLM per gene. Mixed models add random effects for batch or family. All three live in base R plus `lme4`.

- **Linear regression.** Simulate phenotype, genotype, and a covariate, then fit:

```
set.seed(1)
n <- 500
geno <- rbinom(n, 2, 0.3) # 0/1/2 allele count
age <- rnorm(n, 50, 10)
pheno <- 0.3 * geno + 0.02 * age + rnorm(n)

fit <- lm(pheno ~ geno + age)
summary(fit)
confint(fit) # 95% CI for each coefficient
```

  The genotype coefficient should be near 0.3 and significant. This is one row of a GWAS.
- **Tidy output with broom.**

```
library(broom)
tidy(fit, conf.int = TRUE)
glance(fit) # R^2, AIC, BIC, residual SE
```

  `tidy` returns a tibble with one row per coefficient (`estimate`, `std.error`, `statistic`, `p.value`, `conf.low`, `conf.high`). Combine with `dplyr` and you can iterate over thousands of variants with `purrr::map_dfr` and still get a clean data frame out.
- **Residual diagnostics.** `plot(fit)` gives four diagnostic plots: residuals vs. fitted (non-linearity), QQ plot of residuals (normality), scale-location (heteroscedasticity), residuals vs. leverage (outliers). Spend a minute on each; if the QQ plot fans out at the tails, consider transforming or switching to a GLM.
- **GLMs when the outcome is not continuous.** Case/control: `glm(case ~ geno + age, family = binomial)`. Counts: `glm(counts ~ geno, family = poisson)` — and if counts are overdispersed, use `MASS::glm.nb` for a negative-binomial GLM. The `summary` and `confint` calls are identical.
- **Mixed-effects models** for correlated observations (multiple measurements per donor, batches, family structure):

```
library(lme4)
# y depends on treatment, with a random intercept
# per donor:
m <- lmer(y ~ treatment + (1 | donor), data = d)
summary(m)
```

```
confint(m, method = "Wald")
```

You will use these in Running Project Problem Set 2.

You simulated a 500-sample dataset, fit `lm`, `glm` (logistic), and `lmer`, and read coefficients plus 95% CIs from each. Your `plot(fit)` diagnostics show no obvious curvature or outliers on the simulated data (as expected — the data were simulated under the model's assumptions). Break an assumption on purpose and see what the diagnostics show.

Sprint 5: Power Analysis by Simulation

(25 min) "How many samples do I need" is the question every biologist asks and every statistician wishes more people asked *before* running the experiment. Closed-form formulas exist for simple tests, but simulation works for any design.

- **Simple baseline: `power.t.test`.**

  ```
  power.t.test(delta = 0.5, sd = 1,
               sig.level = 0.05, power = 0.8)
  # -> n = ~64 per group
  ```

  Use as a sanity check for the simulation below.
- **Simulate power directly.** The definition is operational: power = fraction of experiments in which the test rejects, given a chosen effect size.

  ```
  set.seed(1)
  sim_power <- function(n, delta, nsim = 1000) {
    rejected <- replicate(nsim, {
      x <- rnorm(n, 0, 1)
      y <- rnorm(n, delta, 1)
      t.test(x, y)$p.value < 0.05
    })
    mean(rejected)
  }
  sim_power(n = 30, delta = 0.5)
  sim_power(n = 60, delta = 0.5)
  ```

  At $n = 30$ per group with $\delta = 0.5$, expect power near 0.48; at $n = 60$, near 0.80.
- **A power grid.** Loop over sample sizes and effect sizes:

  ```
  grid <- expand.grid(n = c(10, 30, 60, 120),
                      delta = seq(0, 1, by = 0.1))
  grid$power <- mapply(sim_power, grid$n, grid$delta)

  library(ggplot2)
  ggplot(grid, aes(delta, power,
                   color = factor(n))) +
    geom_line() +
    geom_hline(yintercept = 0.8, linetype = 2)
  ```

  The result is a family of S-curves — one per sample size — and a horizontal guide at 80%. Pick the smallest n whose curve crosses 0.8 at your expected effect size.
- **Why simulate?** Closed-form power is limited to a handful of designs. Simulation generalizes: unbalanced groups, nuisance covariates, mixed-effects designs, different test statistics, and non-normal outcomes are all just a change to the inner loop.

You produced a power-by-sample-size plot with a 0.8 reference line and can name the sample size needed to detect your chosen effect. Save the plot; you will use the same pattern in Running Project Problem Set 1.

Sprint 6: PCA and Bayesian Shrinkage in R *(25 min)*

- **PCA with `prcomp`.** Load a genotype matrix `G` (rows = samples, columns = variants; for practice use a 1000 Genomes subset or a simulated one).

  ```
  pc <- prcomp(G, center = TRUE, scale. = TRUE)
  str(pc, max.level = 1)
  # pc$x : sample scores (n x k)
  # pc$sdev: stdev per PC
  ```

  `scale. = TRUE` standardizes columns — important when features are on different scales. For MAF-aligned genotypes it is conventional.
- **Plot the first two PCs** colored by a population label:

  ```
  df <- data.frame(PC1 = pc$x[, 1],
                   PC2 = pc$x[, 2],
                   pop = pop_label)
  library(ggplot2)
  ggplot(df, aes(PC1, PC2, color = pop)) +
    geom_point(alpha = 0.7)
  ```

  Continental ancestry typically separates cleanly on PC1–PC2; within-continent structure shows up on PC3–PC5.
- **Scree plot and variance explained.**

  ```
  ve <- pc$sdev^2 / sum(pc$sdev^2)
  plot(ve[1:20], type = "b",
       xlab = "PC", ylab = "variance explained")
  ```

  The "elbow" approximates how many PCs carry signal rather than noise. For GWAS, including the top 5–10 PCs as covariates typically absorbs ancestry and controls population-structure inflation.
- **Bayesian shrinkage in one example.** Empirical Bayes shares information across genes (or tests) to stabilize noisy per-unit estimates. The cleanest demonstration is `limma`:

  ```
  library(limma)
  # expr: gene x sample matrix, design: model matrix
  fit <- lmFit(expr, design)
  fitB <- eBayes(fit) # shrinks per-gene variance
  topTable(fitB, coef = 2, number = 10)
  ```

  `eBayes` borrows strength across genes: genes with few observations get their variance pulled toward the global trend, which stabilizes tiny denominators and reduces false positives. Compare the moderated t-statistic `fitB$t` to the ordinary t `fit$coefficients / fit$stdev.unscaled`. Same idea appears in DESeq2's dispersion shrinkage and in SuSiE's credible sets.
- **"95% credible set" in plain language.** Under the model, there is posterior probability 0.95 that the true causal variant is one of the variants in the set. The set itself is data-dependent, unlike a frequentist confidence interval whose *procedure* has 95% coverage.

Your PC1–PC2 plot shows clear clusters by population label, your scree plot has a visible elbow, and you ran `limma`'s `eBayes` and can articulate in one sentence what shrinking per-gene variance toward a

common trend accomplishes.

End of Blocks 7–8

This is the most conceptually dense part of the curriculum. Take a day or two to let it settle before moving into applications.

Can I move on? Checkpoint

- You can simulate from any common distribution in R and identify which family fits a real dataset
- You can run t-test, Wilcoxon, Fisher's exact, and a custom permutation test, and report effect size plus CI alongside p-value
- You can apply Bonferroni and BH corrections with `p.adjust` and explain the difference
- You can fit `lm`, `glm`, and `lmer`, pull coefficients with `broom::tidy`, and read residual diagnostics
- You can write a power-by-sample-size simulation and read off the required n
- You can run PCA with `prcomp`, interpret the scree plot, and explain `limma`'s eBayes shrinkage as one concrete empirical-Bayes example

Resources

- StatQuest YouTube (Josh Starmer) — visual explanations of most concepts above
- Holmes & Huber – *Modern Statistics for Modern Biology* (free online; all examples in R)
- *ModernDive* (moderndive.com) — introductory statistics with R, free online
- Storey & Tibshirani – Statistical significance for genomewide studies (PNAS 2003; the FDR / q-value paper)
- The `broom` vignette and the `lme4` cheat sheet on the Posit site

Blocks 9–10: Applications — Single-Cell & GWAS

Primary: scRNA-seq pipeline, GWAS pipeline | **Interleave:** All previous skills converge here

Unix, Python, R, and statistics all appear in these two blocks. You will run complete analyses from raw data to biological interpretation.

Block 9: Single-Cell RNA-seq, Start to Finish

Sprint 1: Technology and Data Orientation *(25 min)*

- 10x Chromium captures RNA from individual cells, producing a UMI count matrix with cells as rows and genes as columns in `AnnData`.
- Download a public PBMC dataset from the 10x Genomics website (https://www.10xgenomics.com/datasets); a pre-processed `.h5ad` is fine to start.
- Install scanpy if needed: `mamba install -c conda-forge -c bioconda scanpy`. Docs: https://scanpy.readthedocs.io/. Load with `adata = sc.read_h5ad('pbmc.h5ad')`.
- Explore: `adata.shape`, `adata.obs.head()`, `adata.var.head()`.
- Answer in your journal: what does each row of `adata` represent? Each column? What is a plausible range for per-cell total counts and for number of expressed genes? (For a healthy PBMC droplet library, 500–10,000 genes and 1,000–50,000 UMIs per cell are common.)

You loaded a real single-cell dataset and can describe its dimensions and what each axis means.

Sprint 2: Quality Control: Filtering Cells and Doublets *(25 min)*

- Flag mitochondrial genes so they can be counted:

  ```
  adata.var['mt'] = adata.var_names.str.startswith('MT-')
  ```

- Calculate QC metrics:

  ```
  sc.pp.calculate_qc_metrics(adata, qc_vars=['mt'])
  ```

  This adds `n_genes_by_counts`, `total_counts`, and `pct_counts_mt` to `adata.obs`.
- Plot distributions:

  ```
  sc.pl.violin(adata,
    ['n_genes_by_counts',
     'total_counts',
     'pct_counts_mt'])
  ```

- Filter: keep cells with roughly 200–5000 detected genes and fewer than 20% mitochondrial reads. The biological rationale: very few genes suggests empty droplets or dying cells; very many suggests doublets; high mitochondrial fraction suggests cells whose membranes have leaked cytoplasmic RNA.
- Doublet detection with Scrublet: `sc.pp.scrublet(adata)` (or `sc.external.pp.scrublet(adata)` on older scanpy versions).

You filtered out low-quality cells and doublets. Record cell counts before and after in `journal.md`. A typical PBMC dataset loses 10–30% of droplets.

Sprint 3: Normalize → HVGs → PCA → UMAP → Cluster *(25 min)*

- `sc.pp.normalize_total(adata); sc.pp.log1p(adata)` — library size normalization
- `sc.pp.highly_variable_genes(adata)` — select ∼2000 informative genes
- Run the pipeline:

```
sc.tl.pca(adata)
sc.tl.neighbors(adata)
sc.tl.umap(adata)
sc.tl.leiden(adata, resolution=0.5)
```

- `sc.pl.umap(adata, color=['leiden'])` — a single-cell cluster plot from raw data

You went from filtered counts to a clustered UMAP plot with six scanpy calls. Save the plot to your repo.

Sprint 4: Annotate Cell Types and Pseudobulk DE *(25 min)*

- Plot canonical markers: `sc.pl.dotplot(adata, ['CD3D', 'CD14', 'MS4A1', 'NKG7'], groupby='leiden')`. For PBMCs: `CD3D` marks T cells, `CD14` monocytes, `MS4A1` B cells, `NKG7` NK/cytotoxic cells.
- Assign labels based on which clusters light up for which markers:

```
adata.obs['cell_type'] = adata.obs['leiden'].map(
  {'0': 'CD4 T', '1': 'Monocytes', ...})
```

- For differential expression across conditions, use pseudobulk: aggregate UMI counts per (donor, cell_type) pair and feed the resulting count matrix to DESeq2 or edgeR in R.
- Why pseudobulk? Cell-level tests treat every cell as an independent observation, but cells from the same donor share many sources of variation. Pseudobulk respects the actual unit of replication (the donor) and produces calibrated p-values.

You annotated cell types from marker expression, ran a pseudobulk DE test, and can state in one sentence why pseudobulk gives more honest p-values than per-cell testing.

- CITE-seq: same cells, RNA + protein measurements → Seurat WNN for joint embedding
- scATAC-seq: chromatin accessibility per cell → ArchR, Signac
- Spatial transcriptomics: gene expression + spatial location → Squidpy, cell2location

- Perturb-seq: CRISPR perturbations linked to transcriptomes → Mixscape, scMAGeCK

Block 10: GWAS Pipeline, Variant to Function

Sprint 5: Genotype QC with PLINK2 *(25 min)*

- PLINK2 is the workhorse. Install via bioconda (`mamba install -c bioconda plink2`) or download a pre-built binary from https://www.cog-genomics.org/plink/2.0/. Get a small dataset, e.g., a HapMap or 1000 Genomes subset in PLINK format.
- Sample QC flags: `-mind` (per-sample call rate), `-het` (heterozygosity), `-check-sex`, `-king-cutoff` (relatedness).
- Variant QC flags: `-geno` (per-variant call rate), `-hwe` (Hardy–Weinberg equilibrium), `-maf` (minor allele frequency threshold).
- PCA: `plink2 -pca 10` computes the top 10 principal components. Plot PC1 vs. PC2.

You QC'd a genotype dataset and can report how many samples and variants were dropped by each filter. Every GWAS starts here.

Sprint 6: Association Testing *(25 min)*

- Simulate a phenotype correlated with a chosen causal variant so you know the true answer and can check your pipeline recovers it.
- Run `plink2 -glm` to fit a linear (quantitative) or logistic (case/control) model at every variant.
- For real human data, use REGENIE or BOLT-LMM (mixed models that handle population structure and relatedness simultaneously).
- Make a Manhattan plot and a QQ plot of the results, and check the genomic inflation factor λ_{GC}. Values near 1.00 are healthy; values above 1.1–1.2 suggest uncontrolled structure or miscalibration.

You ran a GWAS on simulated data and recovered the known causal variant as the top hit. Your QQ plot and λ_{GC} indicate a calibrated test.

Sprint 7: Post-GWAS: Heritability, Fine-Mapping, Interpretation *(25 min)*

- LDSC (https://github.com/bulik/ldsc) estimates SNP-heritability from summary statistics: what fraction of phenotypic variance is explained by the common-variant tagging set? Install via conda; follow the repo's instructions for its Python-2 dependency, or use the Python-3 fork `ldsc-py3`.
- Fine-mapping: the SuSiE R package (https://stephenslab.github.io/susieR/) reports 95% credible sets, the smallest sets of variants likely to contain the causal one in each signal. `install.packages("susieR")`.
- Colocalization: the `coloc` R package (`install.packages("coloc")`, https://chr1swallace.github.io/coloc/) tests whether your GWAS signal shares a causal variant with a nearby eQTL.
- Polygenic risk scores (PRS) aggregate small effects across the genome into a single per-sample predictor, typically by weighted sum of effect alleles.

For one locus in your GWAS, you reported a heritability estimate for the trait, a fine-mapped credible set, and a colocalization posterior with an eQTL (or a clear "no colocalization").

End of Blocks 9–10

You've now run the two most important analysis pipelines in modern genomics end-to-end. Take a break before the final unit.

Checkpoint:

- You can run a complete scRNA-seq analysis from raw counts to annotated clusters
- You can explain every step in the single-cell pipeline and why each matters
- You can QC genotype data, run an association test, and make Manhattan/QQ plots
- You can explain heritability, fine-mapping, and colocalization at a conceptual level

Resources

- Scanpy tutorials (scanpy.readthedocs.io) — the PBMC 3k tutorial is the canonical starting point
- Heumos et al. – Best practices for single-cell analysis (Nat Rev Genet, 2023)
- Uffelmann et al. – Genome-wide association studies (Nat Rev Methods Primers, 2021)
- Marees et al. – A tutorial on conducting GWAS (step-by-step, very practical)
- LDSC wiki on GitHub — the definitive usage guide

Blocks 11–12: AI, Right Questions & Capstone

Primary: AI prompting, data expectations, reproducible pipelines, capstone project

> Tools change. Methods change. The ability to ask the right question, recognize unexpected data, and evaluate AI output — that lasts your whole career.

Block 11: Asking the Right Questions + AI Mastery

Sprint 1: The Question Framework *(25 min)*

- Before every analysis, write down answers to: what is the biological question, what would a positive result look like, and what would a null result look like?
- Data inventory: how many rows? Columns? What does each represent? What are the expected ranges?
- Sanity checks: roughly 4–5 million variants per human genome, about 20,000 protein-coding genes, 500–5000 genes detected per cell in scRNA-seq.
- Red flags to watch for: a flat Manhattan plot (no signal at all), bimodal quality score distributions, PCs that correlate with batch instead of biology.

> You have a one-page pre-analysis checklist for the dataset you plan to work with. Commit it to your repo before you touch the data.

The Question Map

Before you start	What is the question? What would a definitive answer look like?
Loading data	What do rows/columns represent? Expected dimensions? Missing values?
Choosing methods	What does this method assume? Does my data meet the assumptions?
After QC	Do distributions look right? Do controls work? Any batch effects?
Interpreting results	Is the effect size meaningful? Could confounders explain this?
Before publishing	Can someone else reproduce this? Are all parameters recorded?

Sprint 2: Prompting AI Like a Bioinformatician *(25 min)*

- Context first: "I have a 10x scRNA-seq h5ad file, 50,000 cells, human PBMCs. I want to . . . "
- Constraints: "Use scanpy 1.9 or newer, avoid deprecated functions, include a comment at each QC threshold explaining the rationale."

- Verification: "Add assertions that check output dimensions and raise a clear error if something is unexpected."
- Debugging: paste the full error message and traceback. Do not paraphrase; paste.

You wrote a single prompt that produced working, well-commented code on the first run, and saved both the prompt and the output to your repo.

Sprint 3: Spotting AI Hallucinations *(25 min)*

- Common failure modes: inventing function names, mixing up package versions, using wrong parameter names, citing papers that do not exist.
- Always test on a small known-answer dataset before trusting the code on real data.
- Cross-check: ask two different AI tools the same question. If they disagree, dig in.
- Version trap: AI outputs frequently mix scanpy 1.8 vs. 1.10 syntax, or old Seurat v3 vs. v5 function names.

Ask an AI tool to "write scanpy code that computes the mean expression of gene *GAPDH* per Leiden cluster from an `AnnData` object." Run the code as-is. Then verify every function name and argument against the current scanpy documentation (https://scanpy.readthedocs.io/). If nothing is wrong, repeat the prompt with a deliberately obscure request (e.g., "use `sc.tl.expression_by_cluster`" — this function does not exist). Note in your journal what the AI did when asked about an invented function.

You have a journal entry documenting one verified, correct AI output and one case where the AI either invented a function or hallucinated confidently about an invented API you asked it to use.

- Always run AI-generated code before trusting it.
- Cite AI assistance in your methods section. Transparency matters.
- Never use AI for novel method development without deep verification.
- AI is a study partner, not a replacement for understanding.

Block 12: Reproducible Pipeline + Capstone

Sprint 4: Snakemake: Your Pipeline in Code *(50 min)*

- Install: `mamba install -c bioconda -c conda-forge snakemake`. Docs: https://snakemake.readthedocs.io/.
- Snakemake defines a pipeline as a set of rules. Each rule has input files, output files, and a shell or Python command that turns one into the other.
- Start simple: a rule to download, a rule to run QC, a rule to align. Run with `snakemake -cores 4`.
- Snakemake figures out the execution order from the rules' inputs and outputs. If you change one input file, it re-runs only what depends on that file.

- Add a `conda:` directive to each rule to pin a per-step environment. At that point the pipeline is fully reproducible on any machine with Snakemake and conda.
- Alternative: Nextflow plus nf-core if you prefer. Install Nextflow with `curl -s https://get.nextflow.io | bash`. nf-co.re (`https://nf-co.re`) hosts community-curated pipelines for most common assays.

You have a multi-step Snakemake pipeline in a Git repo that someone else could clone and run. Verify by changing one input file and confirming only the downstream rules re-run.

Capstone Project

This is your final project. It ties together everything from the past 11 blocks into a single, complete analysis.

- Choose a biological question and a public dataset (GEO, SRA, GWAS Catalog).
- Build a reproducible pipeline (Snakemake or Nextflow) in a Git repository.
- Analyze with Python and/or R, using appropriate statistical methods.
- Produce at least 3 publication-quality figures with captions.
- Write a 5–10 page report: question, methods, results, interpretation.
- Include an AI journal: how you used AI tools, what worked, what didn't.

- Complete Problem Set 4 (QTL Mapping) from the Running Project — this is the recommended capstone if you've been working through Problem Sets 1–3
- Reproduce a published scRNA-seq analysis with updated methods (new normalization, better clustering)
- GWAS-to-function: fine-map a locus, colocalize with eQTLs, identify candidate genes
- Cross-ancestry PRS: build polygenic risk scores, test portability across populations
- CRISPR screen reanalysis: apply MAGeCK + BAGEL2 to a published dataset, compare hit lists

Sprint 5: The Reproducibility Test *(25 min)*

- Clone your repo to a different computer, or ask a colleague or an AI agent to try.
- Can they install the environment and run the pipeline end-to-end from the commands in your README?
- If yes, you are done. If no, fix whatever broke and repeat.

Someone else reproduced your analysis from your repo without asking you for help. That is the standard to aim for.

End of the curriculum

At this point you can process genomic data in Unix, analyze it in Python and R, apply rigorous statistics, and produce reproducible research. The foundation is in place; continued learning comes from applying these skills to real research questions.

Running Project: From Growth Curves to QTL Mapping

This four-part project follows a single scientific narrative: can you find the genetic basis of a yeast strain's response to an unknown compound? Each problem set is designed to be worked on during or shortly after the block that teaches the relevant skills. The problems build on each other; work through them in order.

Clone the data once at the start:

```
git clone https://github.com/joshsbloom/QuantGenSandbox.git
cd QuantGenSandbox
```

A link to a companion Dropbox folder (for the large FASTQ files used in Problem Set 3) is in the repo README. The project culminates in a QTL mapping scan that identifies a candidate gene and generates a hypothesis about the compound's identity.

1. Yeast Growth Curves	Blocks 5–6 (R)	R scripting, ggplot2, t-tests, permutation tests, power simulation
2. Heritability & Mixed Models	Blocks 7–8 (Stats)	Linear models, random effects, heritability, experimental design
3. Short-Read Alignment & Strain ID	Blocks 9–10 (Apps)	Unix pipelines, bwa, sambamba, GATK, BLAST, Hamming distance
4. QTL Mapping	Blocks 11–12 (Capstone)	Genome-wide scans, permutation thresholds, LOD scores, confidence intervals

Problem Set 1: Yeast Growth Curves

Work on this after Blocks 5–6.

Two yeast strains — BY (lab reference) and RM (vineyard isolate) — are grown in YNB + glucose media. You want to determine whether they grow at different rates. 120 µL cultures of each strain were grown overnight to saturation in a 96-well plate (Plate080525_YNB_GLU): BY in well A01, RM in well A02. You will inoculate replicate growth curves using a Biomek liquid-handling robot and then analyze the resulting data.

Part A: Generate the Biomek Randomization CSV

The Biomek robot needs a CSV file specifying: which source well to pipette from, which target well to pipette into, and how much volume. The 96-well plate has rows A–H and columns 01–12. All column numbers less than 10 must have a leading zero (e.g., B03, C09).

Goal: Inoculate 1 µL of saturated culture into 30 random wells for each strain, giving you 30 replicate growth curves per strain. Edge wells (row A, row H, column 01, column 12) should be left as media blanks to control for evaporation and contamination.

The CSV must be comma-delimited with DOS line endings and this header:

`Source.Plate,Source.Wells,Strain,Target.Plate,Target.Wells,Volume`

- Write R code to generate the Biomek CSV file. Use `set.seed(10)` for reproducibility.
- Useful functions: `data.frame()`, `sample()`, `paste0()`, `rep()`, `toupper()`, `sprintf()`, `write.table()` with `eol='\r\n'`.
- Challenge: write the entire script in 4 lines of R.

Part B: Fit Growth Curves & Extract Doubling Times

Read in the plate reader data (`growthCurves_01.csv`) and the Biomek CSV (`randomize_01.csv`) from the `joshsbloom/QuantGenSandbox` GitHub repository, under `projects/growthCurves/`.

- Use the QurvE R package (`growth.gcFitSpline` function) to fit spline models to the growth curves.
- Use lubridate to convert time stamps into decimal hours.
- Extract doubling times during log-phase growth from the fitted models.
- Visualize a few individual growth curves with their fits to confirm the procedure works and the fitting looks reasonable.

Part C: Visualize & Test for Differences

- Use ggplot2 to make a histogram of doubling times by strain (filled by strain).
- Make a violin plot showing individual data points, mean, and 95% confidence interval of the mean for each strain.
- Use `t.test()` to test for a difference in means. Is a paired t-test appropriate here? (Consider: would there be a natural pairing between a specific BY well and a specific RM well?)

Part D: Permutation Test

The assumptions of a parametric t-test may not hold. Perform a permutation test to assess significance without distributional assumptions.

- Generate a null distribution by permuting strain labels 1000 times. For each permutation, compute the t-statistic.
- Visualize the null distribution and mark where the observed (unpermuted) t-statistic falls.
- Calculate a permutation-based p-value.
- Question: What assumptions does the permutation test make? What assumptions of the parametric t-test might be violated here?

Part E: Power Analysis by Simulation

How small a fitness difference can you detect? Evaluate statistical power across a grid of parameters:

- Replicates per strain: 3, 6, 12, 24, 48, 96, 384
- Doubling time means for the new strain: 60 to 240 minutes in 3-minute increments
- Assume BY mean = 90 min, SD = 18 min; the other strain has the same SD

- For each combination: simulate 1000 experiments from two normal distributions with the appropriate means, SD, and sample size.
- Power = fraction of simulations with t-test $p < 0.05$.
- Plot with ggplot2: x = difference in doubling time, y = power, colored by sample size (as a factor).
- Question: If you want to detect a 10-minute difference in doubling time, how many replicates do you need?

Problem Set 2: Heritability & Mixed Models

Work on this after Blocks 7–8.

An unknown compound, chemical X, may inhibit fungal growth. If different yeast strains respond differently, the trait may be heritable, and you could use QTL mapping to find the genetic basis. Eight lab members set up an experiment with ten yeast isolates in YPD and YPD + chemical X across four 96-well plates. A power outage left you with only endpoint OD readings.

Part A: Read & Merge Data

- Read the design file (`02_OD_design.csv`) and the plateR-formatted endpoint OD data (`02_OD_measurements.csv`) from `joshsbloom/QuantGenSandbox` under `projects/modeling-FixedRandom/`.
- Use the plateR R package to parse the plate data.
- Merge measurements with design into a single data frame.

Part B: Visualize

- Make 8×12 heatmaps of OD for each plate using ggplot2 with a viridis color scale. Are there plate- or position-dependent effects?
- Make additional heatmaps showing experimenter and treatment by well position.
- Create a publication-ready 3-panel figure (using patchwork) with panel A twice the size of B and C combined:
- Panel A: Violin plots of endpoint OD by strain with separate violins per treatment. Overlay jittered points, show means, and draw lines connecting YPD vs. YPD:X means within each strain.
- Panel B: Violin + jitter plots grouped by experimenter.
- Panel C: Violin + jitter plots grouped by plate.
- Questions: What do you conclude about strain, treatment, experimenter, and plate effects? Why was it problematic to have different people with different pipettes add strain and media?

Part C: Fixed-Effects Linear Models

- Fit: `OD ~ strain + treatment + plate + experimenter`. What is the average treatment effect? Use `Anova()` to test each factor.
- Add an interaction: `OD ~ strain + treatment + strain:treatment + plate + experimenter`.
- Use a likelihood ratio test to determine whether the interaction term is justified.
- Compare these results to your visualization in Part B.

Part D: Broad-Sense Heritability

- Restrict to YPD:X wells. Estimate broad-sense heritability: $H^2 = \text{Var(Strain)}/(\text{Var(Strain)} + \text{Var(Residual)})$.
- First ignoring experimenter and plate, then including them.
- How do the estimates differ? Why does accounting for these nuisance variables change the heritability estimate?
- Why is heritability of this trait hard to interpret on its own?

Part E: Mixed Model with Random Effects

- Fit: `OD ~ strain * treatment + (1|plate) + (1|experimenter)`. Use lme4 or a similar package.
- Compare to the fixed-effects model using BIC.
- Why are plate and experimenter better modeled as random effects here? Think beyond just the BIC comparison.

Part F: Normalized Response to Chemical X

- Get residuals from a model of OD with random effects of plate and experimenter.
- Compute mean residuals for each strain in YPD.
- Subtract each strain's YPD mean from its residuals in YPD:X. Retain only these corrected YPD:X values.
- Make a violin plot of these normalized strain responses with jittered data points.
- Question: What does this normalization accomplish? What do you conclude about strain-level sensitivity to chemical X?

Part G: Heritability of the Normalized Trait

- Estimate H^2 for the normalized trait.
- Reflect: how can heritability be misleading? How would the results from Parts F and G guide a QTL mapping experiment?

Problem Set 3: Short-Read Alignment & Strain Identification

Work on this after Blocks 9–10.

The lab decided to cross BY to one of the wild isolates and use QTL mapping. To do this, HO endonuclease was disabled in each parent (to maintain haploid state), the strains were mated, the resulting diploid was sporulated, and Heriberto tetrad-dissected over 100 tetrads. DNA sequencing libraries were prepared for each segregant and for the unknown wild parent strain. Unfortunately, no one recorded which wild isolate was used. Your job is to figure it out from the sequencing data.

Data files (FASTQ for the mystery strain and segregants, reference genome, and known variant sites) are available on the linked Dropbox.

Part A: Identify the Mystery Strain

- Manually BLAST a few short reads from the mystery strain FASTQ files against NCBI to confirm they come from yeast.
- Use `bwa mem` to align the paired-end reads to `sacCer3.fasta`.
- Pipe the output through sambamba to convert SAM to BAM and sort.
- Visualize the alignments in IGV. Look at the first 10–20 Kbp on ChrI. What do you see? Does it make sense?
- Use GATK ASEReadCounter with the known variant sites (`mega_filtered.vcf.gz`) to count reference vs. alternate alleles at each site.
- In R: classify sites as alternate (1) or reference (0) based on which allele has more reads.
- Calculate the Hamming distance between the mystery strain's genotypes and each wild isolate in `sparseMatrixGeno1011.RDS`.
- Question: What is the mystery strain?

Part B: Visualize Meiotic Crossovers

- Align reads from the first segregant (seg1 FASTQ files) using the same pipeline.
- Retain genotype calls only at variant sites distinguishing BY from the identified mystery strain.
- Recode: BY variants = 0, mystery strain variants = +1.
- Visualize variant inheritance across the genome.
- By eye: how many crossovers do you see per chromosome?

The following commands chain alignment, format conversion, sorting, and variant counting:

```
bwa index sacCer3.fasta

bwa mem -t 8 sacCer3.fasta \
  reads_r1.fq.gz reads_r2.fq.gz \
  | sambamba view -S -f bam /dev/stdin \
  | sambamba sort -o output.bam /dev/stdin

gatk ASEReadCounter -R sacCer3.fasta \
  -I output.bam -V mega_filtered.vcf.gz \
  -O counts.table
```

Problem Set 4: QTL Mapping

Work on this during Blocks 11–12 as your capstone, or alongside the capstone.

You now have ∼100 haploid recombinant segregants from a cross of BY and the mystery strain you identified in Problem Set 3. Each segregant has been genotyped at marker positions across the genome (encoded as -1 for BY alleles and $+1$ for the mystery strain's alleles) and phenotyped for growth in YPD + chemical X.

Your task is to scan the genome for quantitative trait loci (QTLs) — regions where the inheritance of one parental allele vs. the other is correlated with the phenotype.

Part A: Genome-Wide LOD Scan

For each marker, calculate the Pearson correlation (r) between genotype and phenotype across all segregants. Convert to a LOD score using:

$$\text{LOD} = \frac{-n \ln(1 - r^2)}{2 \ln(10)} \tag{8.1}$$

where n is the number of segregants and r is the Pearson correlation coefficient. The LOD score is a $\log_{10}$ likelihood ratio: a LOD of 2 means the observed linkage is 100× more likely than expected by chance; a LOD of 3 means 1000×.

- Calculate the LOD score at every marker position across the genome.
- Plot the LOD score across genomic position (a LOD scan, analogous to a Manhattan plot). Color by chromosome.
- Which marker has the highest LOD score?

Part B: Permutation Threshold for Significance

A LOD of 2 or 3 as a fixed threshold does not account for the number of markers tested or the correlation structure among nearby markers. Use a permutation procedure to establish an empirical genome-wide significance threshold.

- Permute the assignment of phenotypes to genotypes 1000 times.
- For each permutation, calculate the LOD at every marker and record the maximum LOD across the genome.
- This gives you a null distribution of 1000 genome-wide maximum LOD scores.
- Compare the observed maximum LOD to this null distribution. Visualize the null distribution as a histogram and mark the observed maximum.
- Calculate a permutation-based p-value.
- Question: How could using a simple fixed LOD threshold (e.g., LOD > 2) lead to false positives or false negatives?

Part C: 1.5-LOD Drop Confidence Interval

The 1.5-LOD drop interval is a standard method for defining an approximate 95% confidence interval for the location of a causal variant underlying a QTL.

- Find the peak marker (highest LOD) for the largest QTL.
- Moving outward in both directions from the peak, find the markers whose LOD is ≥ 1.5 units below the peak LOD.
- The interval between these flanking markers is your 1.5-LOD drop confidence interval.
- Report the genomic coordinates (chromosome, start, end) of this interval.

Part D: Candidate Genes & Biological Hypothesis

- List the genes within the 1.5-LOD drop confidence interval. You can use SGD (Saccharomyces Genome Database) or a BED/GFF annotation file.
- Examine the functions of these genes.
- Given what you know from Problem Sets 1–3 about the strains, the experimental setup, and the QTL interval, form a hypothesis: what might chemical X be?
- Write a short paragraph explaining your reasoning.

You just followed the complete arc of a quantitative genetics experiment: phenotyping (growth curves), estimating heritability (mixed models), genotyping (short-read alignment), and genetic mapping (QTL scan with permutation-based significance and confidence intervals).
Each step required a different combination of Unix, R, statistics, and critical thinking. That integration is the point.

Appendix A: Tools & Installation

Every non-default tool referenced in this curriculum, with install commands and official documentation. Most bioinformatics software is installed into a conda environment (see Block 2, Sprint 6); a handful of core tools sit outside conda and should be installed first.

Core infrastructure (install once, before anything else)

- **Miniforge** — conda + mamba pre-configured for the conda-forge channel. Download and run the installer for your OS/architecture from `https://github.com/conda-forge/miniforge`.
- **Git** — macOS: `xcode-select -install` also installs Git as part of the Xcode Command Line Tools. Linux: `sudo apt install git` (Debian/Ubuntu) or `sudo dnf install git-all` (Fedora/RHEL). Docs: `https://git-scm.com`.
- **VS Code** — .dmg (macOS) or .deb / .rpm / tar.gz (Linux) installers at `https://code.visualstudio.com`.
- **Docker** — macOS: Docker Desktop, `https://docs.docker.com/desktop/install/mac-install/`. Linux: the per-distribution Engine install at `https://docs.docker.com/engine/install/`. Rootless alternative: Podman, `https://podman.io`.
- **Apptainer** (formerly Singularity) — HPC clusters usually provide a module (`module avail apptainer`). For a personal Linux install, see `https://apptainer.org/docs/user/main/quick_start.html`. Not supported natively on macOS.
- **R** — install the pre-built binary matching your OS/architecture from CRAN: `https://cran.r-project.org`.
- **RStudio Desktop** (recommended IDE for R) — install after R, free desktop edition: `https://posit.co/download/rstudio-desktop/`.
- **Homebrew** (macOS package manager, useful for everything in the next section) — one-line install at `https://brew.sh`.

Command-line utilities not always installed by default

- `tree` (directory visualization) — macOS: `brew install tree`. Debian/Ubuntu: `sudo apt install tree`. Fedora/RHEL: `sudo dnf install tree`.
- `tmux` (persistent terminal sessions, critical on HPC) — macOS: `brew install tmux`. Linux: `sudo apt install tmux` or `sudo dnf install tmux`. Docs: `https://github.com/tmux/tmux/wiki`.
- `jq` (JSON filtering on the command line) — macOS: `brew install jq`. Linux: `sudo apt install jq`. Docs: `https://jqlang.github.io/jq/`.
- `bc` (arbitrary-precision calculator; used in Block 1 Sprint 4) — usually default on macOS. On minimal Linux images: `sudo apt install bc`.
- `parallel` (GNU parallel, for command-line job parallelism) — macOS: `brew install parallel`. Linux: `sudo apt install parallel`.
- `curl` / `wget` — both default on macOS and most Linux distros; `sudo apt install curl wget` if either is missing.

Bioinformatics tools (install into a conda env)

All of these are available on the `bioconda` channel. Make one environment per project and install what you need:

```
mamba create -n biotools -c conda-forge -c bioconda \
  samtools bcftools bedtools fastp bwa-mem2 star \
  sra-tools plink2 regenie snakemake nextflow
mamba activate biotools
```

Channel order matters: `conda-forge` before `bioconda`. Channel-priority conflicts are the number-one source of conda installation failures. If you hit one, check your `.condarc`.

samtools	View/sort/index/filter BAM & CRAM	`htslib.org`
bcftools	Filter/merge/annotate VCFs	`htslib.org`
bedtools	Intersect, merge, closest genomic regions	`bedtools.readthedocs.io`
GATK	Variant calling (HaplotypeCaller, JointGenotyping)	`gatk.broadinstitute.org`
BWA-MEM2	Short-read DNA alignment	`github.com/bwa-mem2/bwa-mem2`
STAR	Splice-aware RNA-seq alignment	`github.com/alexdobin/STAR`
fastp	FASTQ QC + adapter trimming	`github.com/OpenGene/fastp`
SRA Toolkit (`fastq-dump`, `fasterq-dump`)	Download FASTQ from SRA	`github.com/ncbi/sra-tools`
PLINK 2.0	Genotype QC, GWAS, PCA	`cog-genomics.org/plink/2.0`
REGENIE	Mixed-model GWAS	`rgcgithub.github.io/regenie`
LDSC	Heritability + genetic correlation from summary stats	`github.com/bulik/ldsc`
SuSiE (R package)	Fine-mapping credible sets	`stephenslab.github.io/susieR`
CellRanger	10x Genomics read processing (registration required; not on bioconda)	`10xgenomics.com/support/software/cell-ranger`
Snakemake	Reproducible workflow DAGs	`snakemake.readthedocs.io`
Nextflow	Alternative workflow engine (nf-core pipelines)	`nextflow.io`

R and Python packages

R (CRAN). Inside an R session: `install.packages("tidyverse")` — one command pulls in `dplyr`, `ggplot2`, `readr`, `purrr`, `tidyr`, `stringr`, `forcats`. Also useful: `install.packages(c("data.table", "lme4", "broom"))`.

R (Bioconductor). Inside an R session:

```
install.packages("BiocManager")
BiocManager::install(c("GenomicRanges", "DESeq2",
                       "edgeR", "limma",
                       "SummarizedExperiment",
                       "rtracklayer", "GEOquery",
                       "recount3", "Seurat"))
```

Bioconductor home: https://bioconductor.org.

Python. Install into an active conda env:

```
mamba install -c conda-forge -c bioconda \
  pandas numpy matplotlib seaborn \
  scipy statsmodels scikit-learn \
  pysam biopython scanpy scikit-allel \
  umap-learn
```

The `pip` equivalent works too; prefer conda when a package wraps a C library (`pysam`, `scanpy`) so the pre-built binary is used.

Appendix B: Programming Concepts Cheat Sheet

The concepts you'll use over and over. Bookmark this page.

Dictionaries / hash maps	$O(1)$ lookup — the fastest way to find gene annotations, sample metadata
Vectorized operations	NumPy/R replaces slow loops — essential for million-variant math
Sparse matrices	scRNA-seq is 95% zeros — sparse storage cuts memory 10–100×
Generators	Stream through billion-line files without loading into RAM
Regular expressions	Parse biological IDs, extract annotations, validate formats
Matrix algebra	PCA, linear models, kinship matrices — all matrix operations
Parallel processing	Chromosome-parallel GWAS, multi-sample processing
Error handling	Graceful recovery from malformed records in messy real data
Unit testing	Verify your functions work before running on expensive real data
Documentation	Future-you is your most important collaborator

Appendix C: Reading List (Prioritized)

Start Here (the essentials)

- StatQuest YouTube (Josh Starmer) — watch the PCA, DESeq2, and p-value episodes first
- Software Carpentry: Unix Shell workshop (free, interactive, excellent)
- Scanpy PBMC3k tutorial (the canonical single-cell starting point)
- Marees et al. – A tutorial on conducting GWAS (step-by-step)
- R for Data Science (r4ds.hadley.nz) — free, skip to Ch. 3 if you know basics

Go Deeper (when you're ready)

- Buffalo – *Bioinformatics Data Skills* (the Unix + workflows bible)
- McKinney – *Python for Data Analysis*, 3rd Ed (the Pandas bible)
- Holmes & Huber – *Modern Statistics for Modern Biology* (free online)
- Love, Huber, Anders – DESeq2 paper (Genome Biology, 2014)
- Heumos et al. – Best practices for single-cell analysis (Nat Rev Genet, 2023)
- Uffelmann et al. – Genome-wide association studies (Nat Rev Methods Primers, 2021)
- Bulik-Sullivan et al. – LD Score regression (Nature Genetics, 2015)

Reference (keep on your shelf)

- Gillespie – *Population Genetics: A Concise Guide*
- Efron – *Large-Scale Inference*
- Storey & Tibshirani – FDR paper (PNAS, 2003)
- Wang et al. – SuSiE (JRSS-B, 2020)
- Wilson et al. – Good enough practices in scientific computing (PLOS Comp Bio, 2017)

Online Tools & Databases

- explainshell.com — paste any Unix command to see what each flag does
- Bioconductor workflows — bioconductor.org/packages/release/workflows/
- GWAS Catalog — ebi.ac.uk/gwas/
- PGS Catalog — pgscatalog.org
- nf-core — nf-co.re (community Nextflow pipelines)
- Anthropic Prompt Engineering Guide — docs.anthropic.com

12-Block Overview

0	Setup	Install everything	Git repo with journal.md
1	Unix	Filesystem, pipes, text tools	One-liner pipeline for VCF analysis
2	Unix	Bash, conda/mamba, containers, Git	Automated QC script on GitHub
3	Python	Data structures, parsing, idiomatic Python	FASTA parser with tests
4	Python	Pandas, plotting, bio libs	Manhattan + QQ plots from GWAS data
5	R	Lists, apply family, tidyverse, Bioconductor	GRanges overlaps + data loaded from GEO
6	R	DESeq2, ggplot2	DE analysis + Problem Set 1 (Growth Curves)
7	Stats (R)	Distributions, tests, permutations, multiple testing	QQ plot + BH-corrected calls in R
8	Stats (R)	Linear models, power by simulation, PCA, eBayes shrinkage	Problem Set 2 (Heritability & Mixed Models)
9	Apps	scRNA-seq full pipeline	Annotated cell clusters from real data
10	Apps	GWAS pipeline, post-GWAS	Problem Set 3 (Short-Read Alignment & Strain ID)
11	Meta	AI prompting, right questions	AI research journal
12	Capstone	Pipeline + project	Problem Set 4 (QTL Mapping) or custom capstone